2026年10月 · V261020

OpenAI Codex
从入门到精通

面向工程师与产品经理的Codex全形态实战指南

The Complete Guide to OpenAI Codex — CLI, App, Cloud, Chrome & Beyond

---

适用版本：Codex CLI 0.163.0（本书实测版本）

模型：GPT-6.1 Sol / GPT-6 Astra / GPT-6 Luna

本版：CLI真机截图 · 并入ChatGPT桌面App · 云端可复用环境 · hooks与定时任务

花叔

本手册基于OpenAI官方文档（learn.chatgpt.com/docs）、GitHub仓库openai/codex和本机实测编写。所有操作细节以2026年10月20日的资料为准。AI工具迭代极快，请结合官方文档验证。

本手册半年一修，本版v261020

目 录

Table of Contents

给微信读书读者的话

§ 01 Codex是什么：一个账号，六个入口

§ 02 10分钟开始用Codex

§ 03 你的第一个任务

§ 04 Codex CLI深度使用

§ 05 AGENTS.md：给Codex一张地图

§ 06 桌面App里的Codex：多线程的指挥中心

§ 07 云端Codex：交出去，回来收结果

§ 08 Skills、插件、MCP、hooks、定时任务与/goal

§ 09 从零构建一个完整产品

§ 10 Codex和Claude Code：两个都用时怎么分工

附录A 命令速查表

附录B 定价与套餐

附录C 常见问题

给微信读书读者的话

你好，我是花叔。

2024年底，我用Cursor做了一个叫「小猫补光灯」的iOS App，上线后冲到了App Store付费榜第一。我不会写代码，从来都不会，这个产品从第一行到最后一行都是AI写的。

从那以后，AI编程就成了我每天的工作方式。我用它做产品、写工具、搭工作流，踩了不少坑，也攒了不少经验。这些经验后来变成了你手上这套「AI编程橙皮书」系列。

这是系列的第八本。之前我写了Claude Code入门、源码解析、Harness Engineering、Agent Skills、OpenClaw、Polymarket指南、Gemma4。写了这么多，你可能会觉得我是Claude Code的铁粉。确实是，但我同时也是Codex的重度用户。

AI编程这个领域，2026年最重要的变化之一就是：**不再是一家独大了**。Claude Code和Codex各自走出了不同的路径。Claude Code是终端里的同步对话，你和它面对面协作；Codex是四种形态的异步系统，你把任务丢出去，它在后台帮你搞定。两种思路，两种工作方式，各有各的舒服。

市面上写Codex的内容不少，但大多是单独介绍功能。我觉得少了一个视角：**一个同时深度用Claude Code和Codex的人，怎么看这两个工具，怎么选择，怎么组合**。这本书想提供的就是这个双线视角。

书里所有内容都来自我的真实使用。哪些场景我会开Codex而不是Claude Code，哪些任务适合丢到云端异步跑，哪些时候两个一起用效率最高。这些判断不是看文档能得出来的，是用出来的。

这本书是怎么写出来的

说一件可能会让你意外的事：写橙皮书这件事本身，我现在也大量交给Skill在跑。

过去半年我陆陆续续做了36个Skill，专门解决我每天会反复做的事：选题、调研、审校、配图、做视频脚本、做书、数据分析，各跑各的活。每一个Skill都是我把自己解释了几百次的偏好、踩过几十次的坑，压成的一份SKILL.md。这些Skill装上之后，我每天面对的问题不再是「这件事AI能不能做」，而是「哪件事我还想自己做，哪件事可以让Skill跑掉」。

这本Codex v2的写作过程，就是这套系统的实战。调研是4个subagent并行抓官方changelog、模型层动态、竞品更新、社区反馈，半小时拿回4份结构化报告；写作是4个subagent并行重写章节，每个agent先读花叔的风格规则文件、再下笔；审校是独立的agent跑三遍（AI腔检测、事实核查、跨章节连贯性）；构建是huashu-book-pdf这个skill一条龙输出EPUB / HTML / PDF。我自己干的事是想清楚要写什么、看产出是不是像我说的话、把不像的删掉。整本书从启动调研到出三种格式，前后大约5个小时。

讲这件事不是炫技。是因为Codex本身也有Skills系统（§08会展开讲），而且SKILL.md在2026年已经成了跨agent的事实标准：Codex、Claude Code、Cursor、Gemini CLI共用同一种格式。这个标准化意味着你在一个

工具上写的Skill能在另一个工具上跑，你的工作方式从此可以脱离任何一家公司的产品。这是我看Codex这本书的一个底层视角：工具会换、模型会变，但你为自己积累的Skill是真正属于你的。

如果你从没用过AI编程工具，这本书能带你从零开始。如果你已经是Claude Code用户，这本书能帮你打开另一个维度。如果你已经在用Codex，这本书也许能给你一些新的组合思路。无论哪一种，希望你看完这本书也会想自己造一个Skill试试。

希望这本书对你有用。

v1 → v2修订说明（2026年5月）

这本书v1是2026年4月完稿的。一个月之后我决定做v2，原因很直接：AI编程工具一个月就能大变样，橙皮书不能停在原版本。

这一个月里，Codex发生了几件事：GPT-5.5空降默认模型，输出token减40%但单价翻倍；2026-05-07 Codex for Chrome扩展上线，形态从四种变五种；Auto-review让审批模型从「事事问人」切到「reviewer agent判断」；ChatGPT套餐增加$100 Pro一档，独立开发者多了个甜蜜点；桌面App五件套大改（Computer Use / In-App Browser / Memory / Image Generation / Automations续跑）。这些都不是小修小补，是骨架级变化。

v2改了什么：模型章节、定价附录、形态介绍、沙箱审批这四块基本重写；其他章节做了模型名刷新、命令速查增补、FAQ加5条用得久才会遇到的坑。变化最大的是§01、§10和附录B这三章，老读者直接跳读就能感受到。

这是橙皮书系列第一次明确做版本迭代。之前几本要么写完就定稿，要么悄悄改两处。这次我决定把节奏定下来：**半年一修**，每次修订独立标版本号、附修订说明。AI工具的迭代太快，停在某个时间点的书不如一本会跟着工具一起长的书。

临门补一笔（2026-05-14）：v2定稿没几个小时，OpenAI上线了「Work with Codex from anywhere」，把Codex集成进了ChatGPT手机端。手机现在可以镜像桌面Codex上正在跑的工作并远程审批。我在§01末尾加了「尾声」一节，§06也补了一段。「五形态」的框架仍然成立，手机版是桌面App的远程伴侣，不算独立的第六形态。这种节奏就是AI工具的日常。

10月补记

按「半年一修」，这是第二次修订。这半年Codex的变化比上一次还大：7月9日它并进了ChatGPT桌面App，Work和Codex共用一份订阅和用量；GPT-5.5在10月14日对ChatGPT登录的Codex退役，现在是GPT-6.1 Sol、GPT-6 Astra、GPT-6 Luna三个；CLI从0.130走到0.163，5月版里反复出现的`--full-auto`和讲登录时用过的`--api-key`都删掉了；云端换成了先搭可复用环境、再起任务的做法；hooks、插件、定时任务都成了正式功能。

这一版CLI的部分是在我电脑上真跑出来的。0.163.0装好、登录、建项目、让它做一个任务、看diff、查`/status`，每一步都截了图放进§02到§04。桌面App和云端这次没能上手跑，那两章按官方文档和changelog写，章节里也写明了。

有几处5月版写错了，在这里统一交代：

第一，§10引用的6852个会话的分析，我写成了Stella Laurenzo发在一个Substack上的长文。数据其实来自她4月2日在anthropics/claude-code提的GitHub issue，那篇Substack是别人写的评论。那一节讲的是3、4月Claude Code的情况，两边模型都已经换了好几代，这一版整节删掉了，§10按10月的模型和功能重写。

第二，§04和附录C说Auto-review「4月30日上线」，引了「约99%的低风险动作自动通过、阻断99.3%的prompt injection攻击」两个数，没写出处，后面紧跟着引的是《Running Codex safely at OpenAI》这篇博客。这篇博客里没有这两个数，它们出自4月30日alignment.openai.com的评测文章，含义也和我写的不一样：99.1%是OpenAI内部真实越界请求的批准率，99.3%是在一套人工构造的注入用例上的拒绝率。5月版还说新版CLI默认就是Auto-review，其实默认仍是由你来批准。§04已经按原文改过。

第三，§08说Anthropic至今没做`/goal`，错了，Claude Code 5月11日的2.1.139就加了；§10把hooks列成Claude Code的独有项，Codex的hooks 5月14日也正式上线了。附录B里Opus 4.7的价格写成了每百万token 15/75美元，那是Opus 4和4.1的价，4.7是5/25美元。§10、附录B和附录C里Codex实际上下文「约258K」的说法来自一个论坛帖子，这一版改成以`/status`里显示的为准。

第四，5月版的这封信里说Codex是「四种形态的异步系统」，补笔里又说手机不算独立的形态。现在手机上可以直接从发布好的环境起云端任务，§01按六个入口来讲。Claude Code那边也早就不只是终端里的同步对话了，这半年两家的功能越来越像，§10有对照。

花叔
2026年5月（v2.0.0）
2026年10月补记（v261020）

§01 Codex是什么：一个账号，六个入口

One Account, Six Entry Points

Codex是一套系统：同一个ChatGPT账号、同一份配置、同一套AGENTS.md，从终端、桌面、云端、编辑器、浏览器、手机六个入口进去。这一章讲清楚每个入口是什么，再给你一个起步建议。

AI编程工具的三次换代

拉远一点看全景。AI编程工具这几年变了三次，每次变的是你和AI的分工。

第一次：补全（2022）。GitHub Copilot出来，你写上半句，它猜下半句。写代码的人还是你。

第二次：对话（2023-2024）。Cursor火了。你说想要什么效果，它在编辑器里替你改。后来加了Agent模式，能跨文件操作、跑命令，但它始终长在IDE里。

第三次：agent（2025-2026）。Claude Code和Codex CLI先后出现。它们直接在终端里跑：你描述一个需求，它自己读代码、写代码、跑测试、操作Git。你从写代码的人，变成了做决策的人。



到2026年，agent这条赛道上最主要的两家是Anthropic的Claude Code和OpenAI的Codex。两家都已经不止一个入口了。

2026年10月的Codex长什么样

这半年有三件大变化：

- **Codex并进了ChatGPT桌面App。**2026年7月9日起，原来独立的Codex App成了ChatGPT桌面App（macOS、Windows）的一部分，App里分Chat、Work、Codex三块。8月11日又出了Linux版预览。
- **模型整代换了。**本书5月版的主力GPT-5.5、GPT-5.4，对用ChatGPT账号登录的Codex已经全部下线（5.4在8月31日，5.5在10月14日）。现在的主力是GPT-6.1 Sol、GPT-6 Astra、GPT-6 Luna，CLI的模型列表里排第一的是GPT-6.1 Sol，也是本书实测时的默认。
- **官方文档搬家了。**`developers.openai.com/codex`现在会跳转到`learn.chatgpt.com/docs`，书里的链接都按新地址写。

本书的实测环境是Codex CLI 0.163.0（2026年10月15日发布），截图都来自这台机器。

六个入口

1. Codex CLI（终端）

开源（Apache-2.0），Rust写的，跑在你本地。终端里输入`codex`，进入一个全屏界面（TUI），输入框在底部，上面是对话和操作记录。沙盒分三档：read-only、workspace-write、danger-full-access。这半年CLI从0.130走到0.163，加了worktree、用量面板、可搜索的设置面板、按键重映射、对话中途直接改方向，§04细讲。

适合：想完整看到每一步、喜欢终端工作流、要写自动化脚本的人。

2. ChatGPT桌面App里的Codex

以前叫Codex App，现在是ChatGPT桌面App里的Codex一栏。它能同时开多个线程、每个线程选在本地项目里跑还是开一个新的worktree跑，还带Computer Use（操作你电脑上的其他App）、内置浏览器、图片生成、Memories、定时任务（Scheduled tasks）。§06细讲。

适合：同时推进好几件事、需要它操作本机App、要定时跑任务的人。

3. Codex Cloud（云端）

任务在OpenAI的云端环境里跑，你电脑关机它也照跑。现在的做法是先建一个「可复用的云端环境」：Codex自己检查仓库、装依赖、跑测试、出一份设置报告，你看过后点Publish，之后的任务都从这个环境起跑。旧版的云端设置页还能用，官方标成Legacy，说了计划弃用。§07细讲。

适合：需求说得清、交代完就不用管的活，比如修一个issue、补测试、审PR。

4. IDE扩展

VS Code、Cursor、Windsurf里装扩展（ID是`openai.chatgpt`）；JetBrains和Xcode是在它们自带的AI功能里选Codex。在编辑器侧边栏和它对话，也能把任务丢到云端。

适合：不想离开编辑器的人。

5.浏览器扩展

5月出的Codex for Chrome，现在也支持Edge、Brave、Opera、Vivaldi。它复用你浏览器里已登录的会话，能操作要登录才能看的网页。早期每个线程会自动建一个标签组，5月21日起改成在标签图标上显示状态，不再建标签组。

适合：网页里的重复事务，比如整理后台数据、处理工单。

6.手机：Codex Remote

ChatGPT手机App（iOS、Android）里的Codex Remote，6月25日正式上线。手机扫码和一台跑着Codex的Mac或Windows电脑一对一配对，之后能在手机上看线程进度、审批命令、追加指令；也可以不连电脑，直接选一个云端环境起任务。代码和凭据留在电脑上，手机是遥控器。

六个入口对比

入口	跑在哪	电脑要开着吗	最适合
CLI	本地终端	要	终端用户、脚本和CI
桌面App里的Codex	本地（可选worktree）	要	多线程并行、Computer Use、定时任务
Cloud	OpenAI云端环境	不要	异步委派、代码审查
IDE扩展	编辑器侧边栏	要	不想离开编辑器
浏览器扩展	你的浏览器	要	需要登录态的网页任务
Codex Remote	手机遥控电脑或云端	连电脑时要	离开工位时看进度、审批

这几个入口共享同一个ChatGPT账号和用量额度，本地的几个入口共用`~/.codex/config.toml`和AGENTS.md。你在CLI里配好的MCP服务器，在App和IDE扩展里也能用。

和Claude Code有什么不同

Claude Code也不是只有终端：它有桌面版、网页版（claude.ai/code）、VS Code和JetBrains扩展。两家都是多入口的AI编程系统，功能清单这半年越来越像，差别更多在模型、默认行为和生态侧重。§10专门讲，这一章只讲Codex的入口怎么选。

花叔的入口建议

六个入口不用全开。我的建议是：

先从CLI开始。装完进一个项目目录，让它做一件小事，走完一次从提需求到看结果的循环。CLI能看到它的每一步，最适合建立第一手感觉。

被具体痛点卡住了，再开对应的入口。想让它在你合上电脑以后还在跑，开Cloud；想同时推进三件事、让它定时跑，开桌面App；想让它处理要登录的网页，装浏览器扩展；出门时想看进度，配手机。

> **建议：**每多开一个入口，就多一套要维护的设置和一份要盯的进度。Codex是一套系统，但你不用一上来就把整套系统都装上。

§02 10分钟开始用Codex

Get Started in 10 Minutes

CLI、桌面App、云端、IDE扩展各有各的装法。不用全装，挑一个最顺手的开始。

前提：一个ChatGPT账号

Codex用ChatGPT账号登录，用量算在你的套餐里。Plus（20美元/月）就能用CLI、IDE扩展、网页、手机和云端集成。Free和Go套餐现在也有Codex，但官方写的是在桌面App里用GPT-6 Luna，还在逐步放量。各套餐的细节见附录B。

你也可以用OpenAI的API Key登录，按token付费；这样CLI、SDK、IDE扩展能用，云端那一套（GitHub代码审查、Slack集成等）用不了。

方式一：安装CLI

官方现在首推独立安装器，不需要先装Node.js：

```
# macOS / Linux
curl -fsSL https://chatgpt.com/codex/install.sh | sh

# Windows (PowerShell)
irm https://chatgpt.com/codex/install.ps1 | iex
```

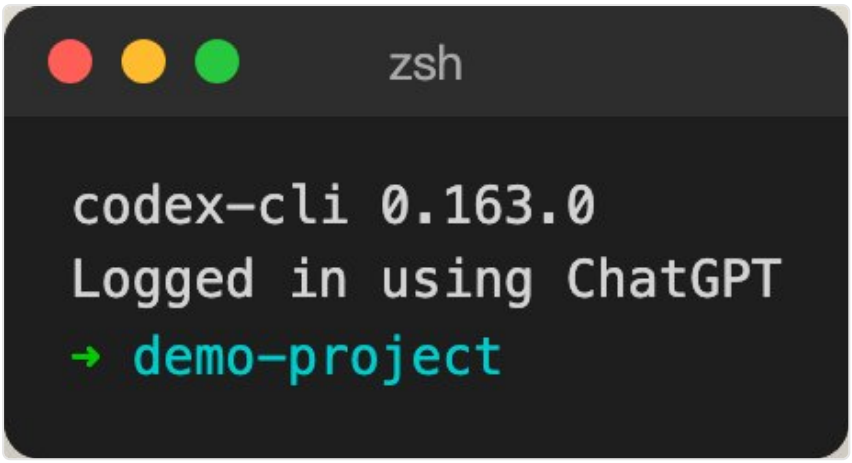
习惯用包管理器的话，也可以：

```
# npm (需要Node.js)
npm install -g @openai/codex

# Homebrew (注意是cask)
brew install --cask codex
```

装完先看一眼版本和登录状态：

```
codex --version
codex login status
```



> 本书实测用的版本：codex-cli 0.163.0，用ChatGPT账号登录。

之后升级用`codex update`，装得有问题用`codex doctor`体检，它会分环境、配置、桌面App、更新、连通性几组检查，把警告汇总在最上面。

登录。第一次运行`codex`会提示登录，默认弹浏览器走ChatGPT授权。没有浏览器的服务器上，用设备码：

```
codex login --device-auth
```

用API Key登录，要通过标准输入传进去：

```
printenv OPENAI_API_KEY | codex login --with-api-key
```

5月版书里写的`codex --api-key sk-...`已经不存在了，现在会直接报错。

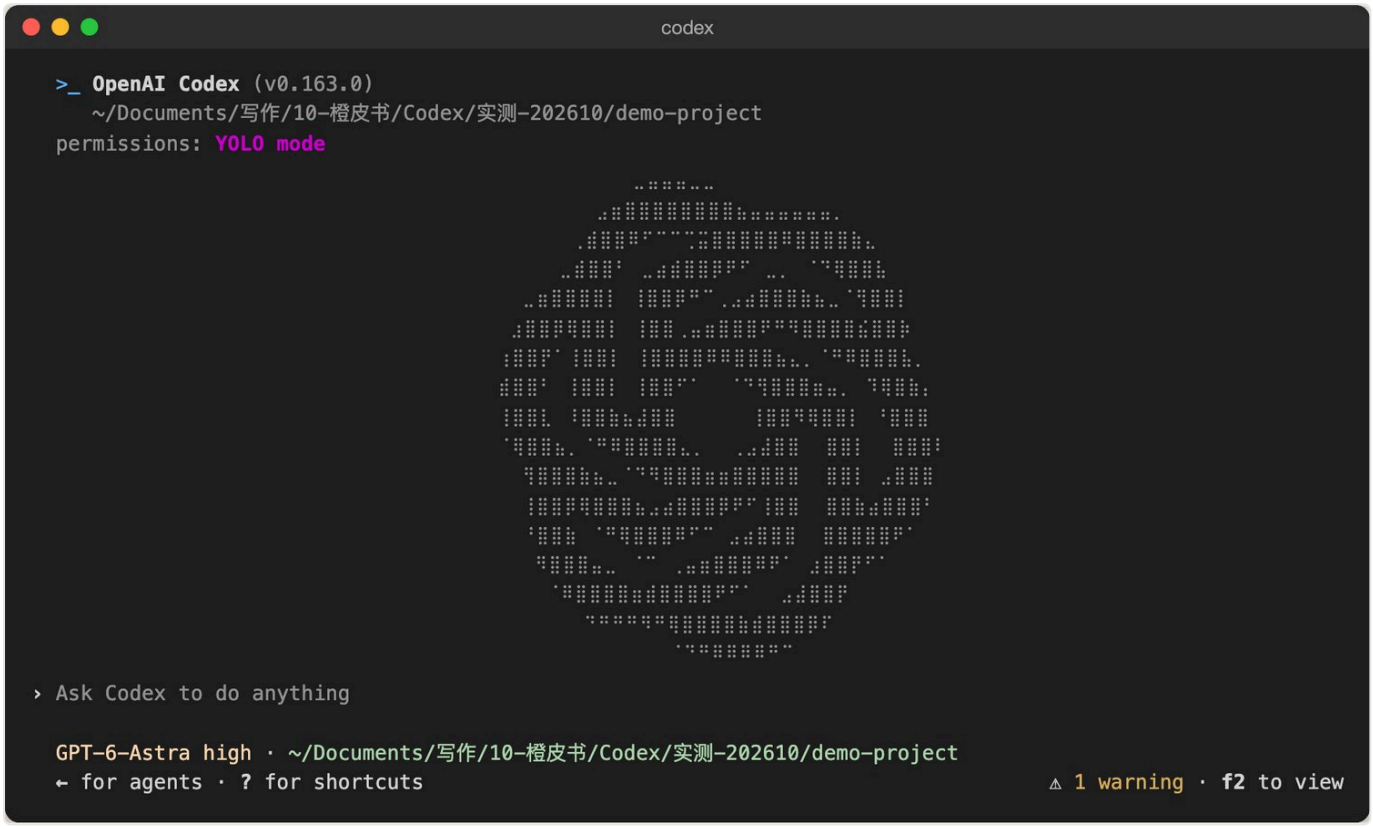
第一次进入项目目录

进到一个项目目录，输入`codex`。如果这个目录Codex没见过，它会先问你信不信任这个文件夹：



> 首次进入新目录的信任确认。原文的意思是：信任之后，Codex可以在你的权限设置范围内读、改、运行这里的文件；文件夹里的设置（比如hooks）可能不经模型就自动执行代码，所以只信任你认识的项目。选择会被记住。

选Trust and continue，就进入了全屏界面。它默认占满整个终端，底部是输入框和状态栏（模型、推理档位、当前目录），上面是对话和它的操作记录。想保留终端原来的滚动历史，启动时加`--no-alt-screen`，或者用`/tui`切到Scrollback模式，下次启动生效。



首屏。注意左上角的「permissions: YOLO mode」和状态栏的「GPT-6-Astra high」是我这台机器自己改过的配置，不是默认值；默认是GPT-6.1 Sol、Ask for approval（§03的截图是默认配置）。

方式二：桌面App

Codex现在在ChatGPT桌面App里。macOS、Windows都有，Linux（Ubuntu、Debian、Fedora，x64和ARM64）是预览版，给的是.deb和.rpm安装包。

已经装了CLI的话，最省事的是：

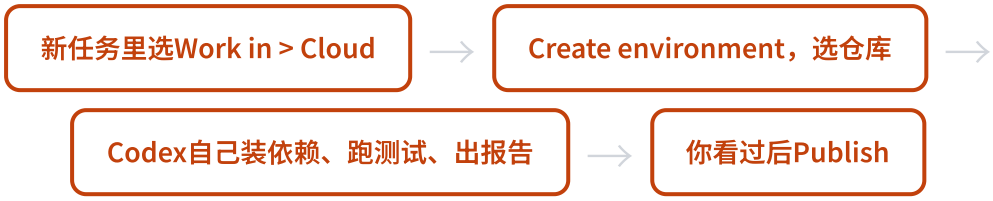
```
codex app
```

它会打开桌面App，没装就打开安装程序。Windows也可以从Microsoft Store装，Windows版带原生沙盒和PowerShell支持，WSL仍然能用，但不再是唯一的推荐路径。

打开App，登录同一个ChatGPT账号，切到Codex，选一个项目文件夹就能开始。用过Claude Code的话，App里的Settings > Import能把Claude Code、Claude Cowork、Cursor里的设置、项目指令和近期会话导进来，CLI里对应的命令是`/import`。导入会改你的Codex配置、把旧会话加进历史，本书实测时在CLI里按下回车就直接导入了，没看到确认，动手前先备份`~/.codex/config.toml`。

方式三：云端

云端不需要在本机装任何东西，但第一次要建一个环境：



环境发布以后，新任务都从这个准备好的环境起跑，你关掉浏览器、合上电脑，它照样在跑。GitHub之外，8月起也支持GitLab（Beta）。§07细讲。

方式四：IDE扩展

VS Code、Cursor、Windsurf、VS Code Insiders里，在扩展市场搜Codex，扩展ID是`openai.chatgpt`。JetBrains用户在IDE自带的AI Chat里选Codex，Xcode用户在Apple的coding intelligence设置里选Codex。

装好后Codex出现在编辑器侧边栏，用ChatGPT账号登录就能对话，它能读写当前项目、跑命令，也能把任务交给云端。

配置文件

本地的几个入口共用一份配置：

```
~/.codex/config.toml
```

常改的几项：

```
# 默认模型
model = "gpt-6.1-sol"

# 推理强度：low / medium / high / xhigh，另有max、ultra
model_reasoning_effort = "medium"

# Web搜索：cached（默认）/ indexed / live / disabled
web_search = "cached"
```

0.163.0起，大部分界面相关的设置不用手改文件了，在会话里输入`/config`就有一个可搜索的设置面板（§04有截图）。想按场景分几套配置，用`--profile 名字`，它会在基础配置上叠加`~/.codex/名字.config.toml`；5月版书里的`[profiles.xxx]`写法从0.134.0起不再支持。

选择建议

你的情况	从哪开始	原因
从没用过AI编程工具	CLI	一行命令装好，每一步都看得见
用过Claude Code	CLI，再试 `/import`	体验最接近，设置能带过来
想同时跑几件事、定时跑	桌面App	多线程、worktree、Scheduled tasks都在这
不想在本机装东西	云端	浏览器里建环境就能用
重度VS Code用户	IDE扩展	不用切换窗口

花叔的建议：拿不定主意就从CLI开始。花5分钟装好，进一个项目目录，让它做一件小事，走完一次从提需求到看结果的循环。再决定要不要开App或云端。

§03 你的第一个任务

Your First Task

用一个真实跑过的小任务，走一遍Codex干活的完整过程：读文件、改代码、请求权限、自己验证、交总结。每一步都有截图。

第一个任务挺重要，它决定了你接下来愿不愿意继续用这个工具。所以这一章不追求复杂，只追求完整：从提需求，到它交回一个能用的结果。

准备一个小项目

我准备了一个很小的目录`demo-project`，里面三个文件：

- `index.html`：一个单文件网页「小猫的备忘录」，一个标题、两条待办，白底黑字。
- `README.md`：一句话说明。
- `AGENTS.md`：给Codex的三条规矩。

```
# AGENTS.md

- 所有回复使用中文。
- 修改页面时保持单文件 HTML，不引入外部依赖。
- 改完后在总结里列出改动的文件名。
```

AGENTS.md是Codex每次开工前都会读的项目说明，§05整章讲它。这里先放三条，看它遵不遵守。

为了还原产品默认的样子，我启动时显式指定了默认模型和默认权限（你自己用时直接输`codex`就行）：

```
codex -m gpt-6.1-sol -c model_reasoning_effort=low \
  -s workspace-write -a on-request
```

提需求

在输入框里写需求，按回车：



> 需求写在底部输入框。状态栏显示当前是GPT-6.1-Sol、推理档位low。输入框下方的「tab to queue message」是说：它干活时你按Tab可以先排一条后续指令。

需求写得具体一点：放哪、怎么交互、用什么记住、有什么限制。它不用猜，你也好验收。

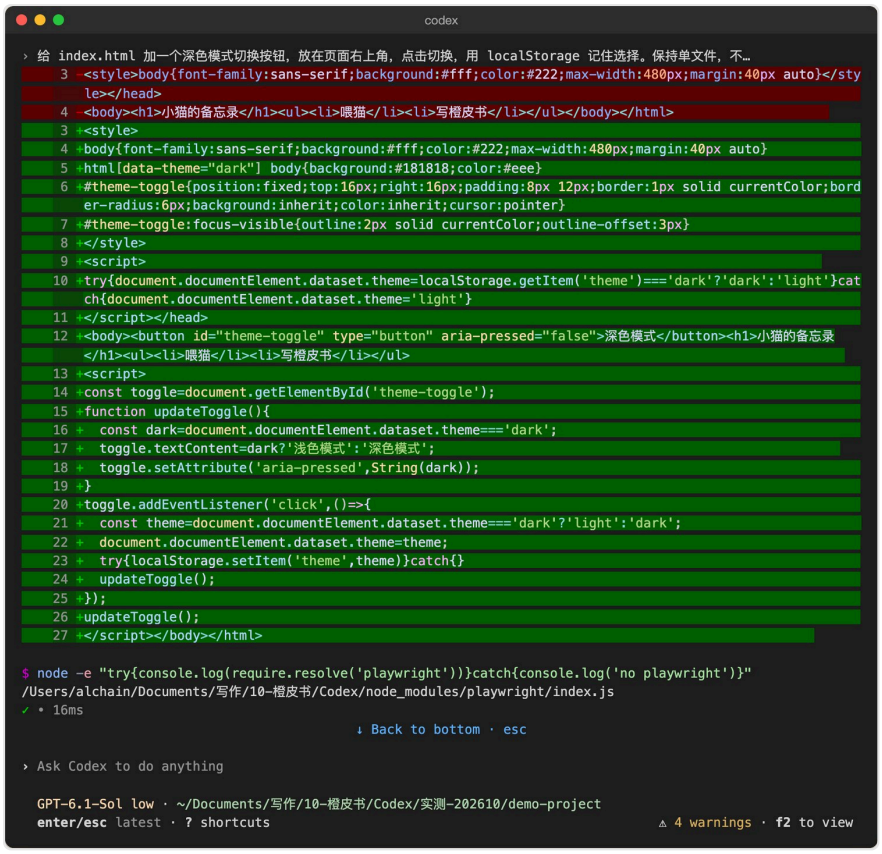
它先读，再改

回车之后，它没有直接动手。先说了一句打算，然后找文件、读文件：



> 按Ctrl+T打开完整的转录视图能看到这一段：先用`rg --files`找到AGENTS.md和index.html，再`cat index.html`看现状，说明自己的处理方式（浅色仍是首次打开的默认），然后才开始改。

改动以diff的形式出现在对话里，红色是删掉的行，绿色是加上的行：

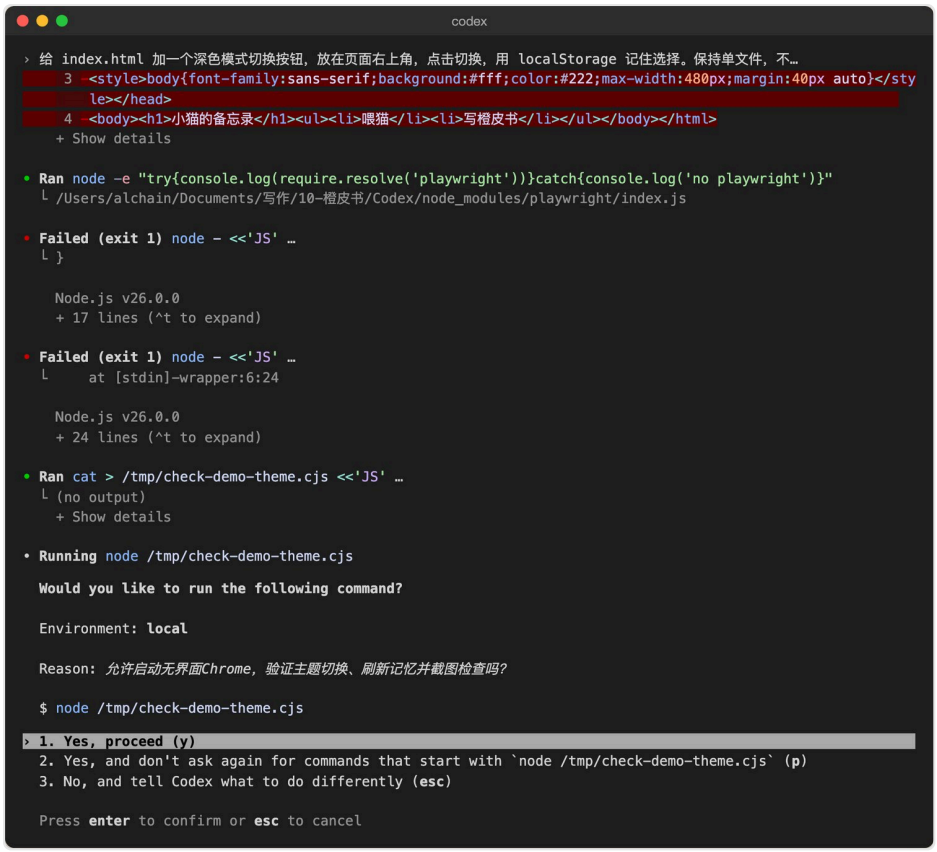


> Edited index.html (+25 -2)。新增了深色主题的样式、固定在右上角的按钮、页面加载时读localStorage的脚本，以及点击切换的逻辑。改工作区里的文件，在默认权限下不用你批准。

不需要逐行看懂。大致扫一眼它改了哪些地方、有没有碰你不想让它碰的东西，就够了。

越界的动作，它会停下来问你

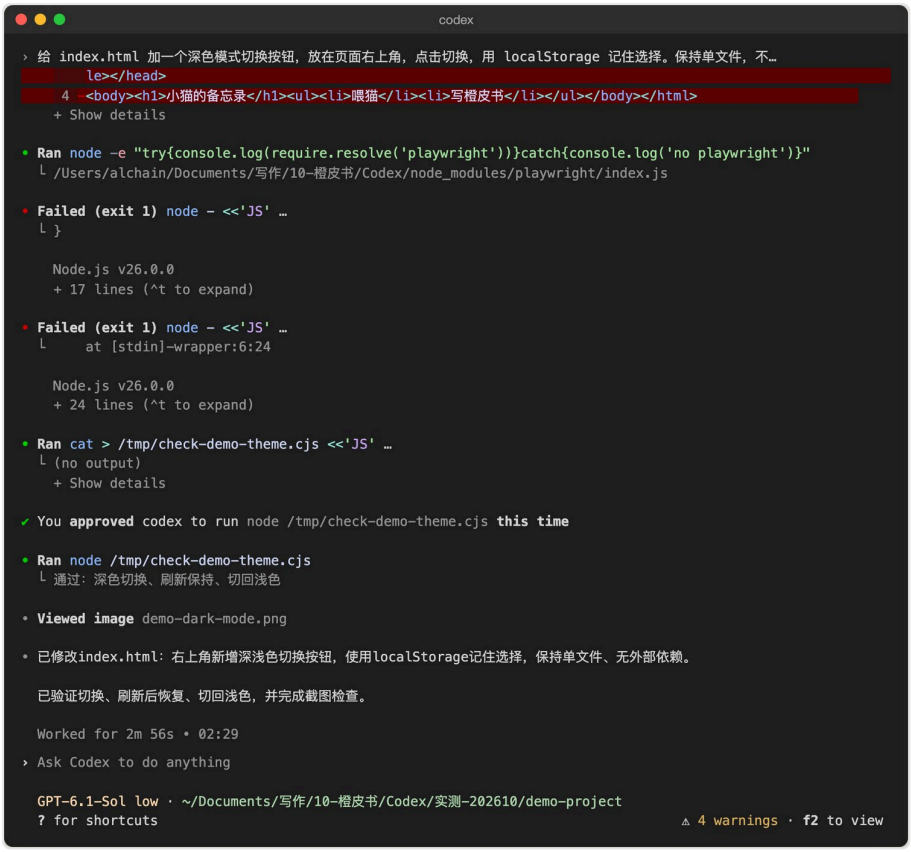
改完代码，它想自己验证一下：写一个小脚本，启动一个无界面的Chrome，打开页面点按钮、刷新、截图。启动浏览器超出了默认沙盒的范围，于是它停下来请求批准：



> 审批弹窗写明了要跑的命令和理由。三个选项：只这一次放行（y）；以后以这个命令开头的都不再问（p）；拒绝，并告诉它换个做法（esc）。上面两条红色的Failed是它前两次写验证脚本出错，自己改了重来。

默认的权限是这样分的：工作区里读文件、改文件、跑普通命令，它自己来；要联网、要改工作区以外的东西、要做沙盒拦住的事，先问你。我按了y，只放行这一次。

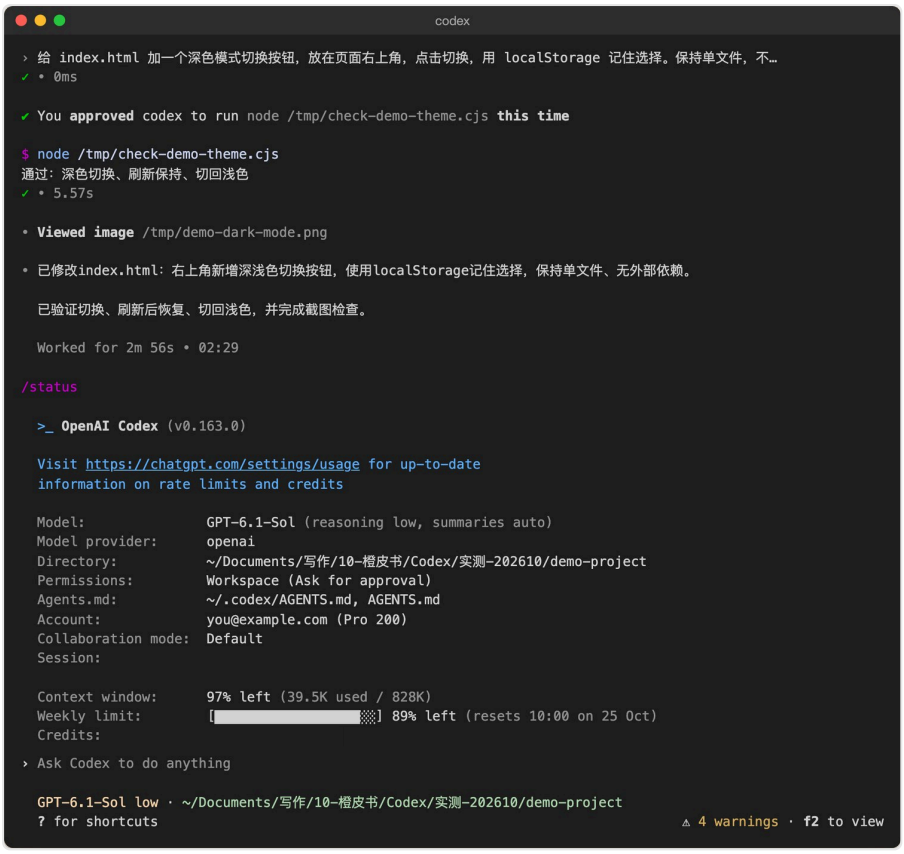
它自己验证，然后交总结



> 验证脚本的输出是「通过：深色切换、刷新保持、切回浅色」，它还看了一眼自己截的图。最后的总结用的是中文、列出了改动的文件名，AGENTS.md里那三条都做到了。

整个任务用了2分56秒，其中大约1分钟是我这边网络重连。最后一行「Worked for 2m 56s」是它报的总时长。

任务结束后输入`/status`，能看到这次会话的配置：



> /status里的几行：模型GPT-6.1-Sol、推理low；权限Workspace（Ask for approval），也就是默认；Agents.md一行列出了这次读到的两份AGENTS.md（全局一份、项目一份）；上下文用了39.5K。会话ID和积分数我遮掉了。

回头看这个循环

这一个小任务里，Codex的工作方式都出现了：

阶段	它做了什么	你要做什么
理解	找文件、读现状、读AGENTS.md	把需求写具体
修改	在工作区里直接改，diff给你看	扫一眼改动范围
越界	停下来，写明命令和理由	决定放不放行
验证	自己写脚本跑、看截图	打开页面亲手点一下
汇报	按AGENTS.md的要求总结	不满意就接着说

想先看计划再让它动手，输入`/plan`切到Plan模式，它会先列方案、等你确认。默认模式下它会边说边做，就像上面这样。

接着往下做

不满意就直接说，比如：

```
按钮在手机上太大了，缩小一点；深色模式下链接颜色看不清，调亮。
```

它会接着改、接着验证。你也可以在它干活的时候直接打字回车，插一句新的要求，它会在当前这一轮里调整方向；按Tab则是排到这一轮结束后再做。

满意了，让它提交：

```
初始化git仓库，把改动提交，commit message写清楚加了什么。
```

建议：第一个任务挑小的。一个页面加个按钮、一个脚本加个参数，十分钟内能看到结果的那种。先把这个循环跑顺，建立信心，再往大里做。

常见问题

它说没法联网怎么办？默认沙盒里命令是断网的，需要装包、访问API时它会请求批准。频繁被问嫌烦，用`/permissions`切到Approve for me，让自动审查替你判断低风险的请求（§04）。

生成的代码有bug怎么办？把报错原样贴给它：「运行后报了这个错误：……」。它会读错误、找原因、改代码。

执行一半想叫停？按Esc打断当前这一轮，再重新说你想要什么。

§04 Codex CLI深度使用

Codex CLI Deep Dive

模型和推理档位怎么选、权限怎么给、全屏界面里有哪些面板、怎么写自动化脚本。这一章的命令和截图都来自本机的Codex CLI 0.163.0。

命令行速查

下面这张表的参数都在0.163.0的`codex --help`里核过。完整的子命令和斜杠命令清单在附录A。

命令	用途	示例
`codex`	启动交互界面	`codex`
`codex "prompt"`	带着第一句话启动	`codex "解释这个项目的结构"`
`-m, --model`	指定模型	`codex -m gpt-6-luna`
`-c key=value`	临时覆盖一项配置	`codex -c model_reasoning_effort=high`
`-s, --sandbox`	沙盒档位	`codex -s read-only`
`-a, --ask-for-approval`	什么时候问你（on-request / never）	`codex -a never -s read-only`
`--approve-for-me`	审批请求先交给自动审查	`codex --approve-for-me`
`-i, --image`	附带图片	`codex -i bug.png "这个报错怎么修"`
`-C, --cd` / `--add-dir`	指定工作目录/追加可写目录	`codex -C frontend --add-dir ../backend`
`--worktree`	在一个新的受管worktree里开会话	`codex --worktree`
`--search`	开实时Web搜索	`codex --search`
`-p, --profile`	叠加一套命名配置	`codex -p work`
`codex exec`	非交互执行	`codex exec "跑测试，修掉失败的"`
`codex review`	非交互代码审查	`codex review --uncommitted`
`codex resume` / `fork`	恢复/分叉之前的会话	`codex resume --last`
`codex agents`	浏览后台服务上的所有会话	`codex agents`
`codex update` / `doctor`	自更新/体检	`codex doctor`

这些旧写法已经失效

如果你照着5月版或者网上的旧教程敲命令，下面几个会直接报错：

```
codex

error: invalid value 'untrusted' for '--ask-for-approval <APPROVAL_POLICY>'
  [possible values: on-request, never]

error: unexpected argument '--api-key' found
Run commands within a Codex-provided sandbox

Usage: codex sandbox [OPTIONS] [COMMAND]... [COMMAND]

Commands:
  uninstall  Remove the legacy Windows sandbox's machine-wide accounts and network rules
  help       Print this message or the help of the given subcommand(s)

Arguments:
  [COMMAND]...
          Full command args to run under seatbelt

→ demo-project
```

`-a untrusted` 报错，现在只接受on-request和never；`--api-key` 不存在；`codex sandbox` 的子命令只剩uninstall。

```
zsh

error: unexpected argument '--full-auto' found

  tip: to pass '--full-auto' as a value, use '-- --full-auto'

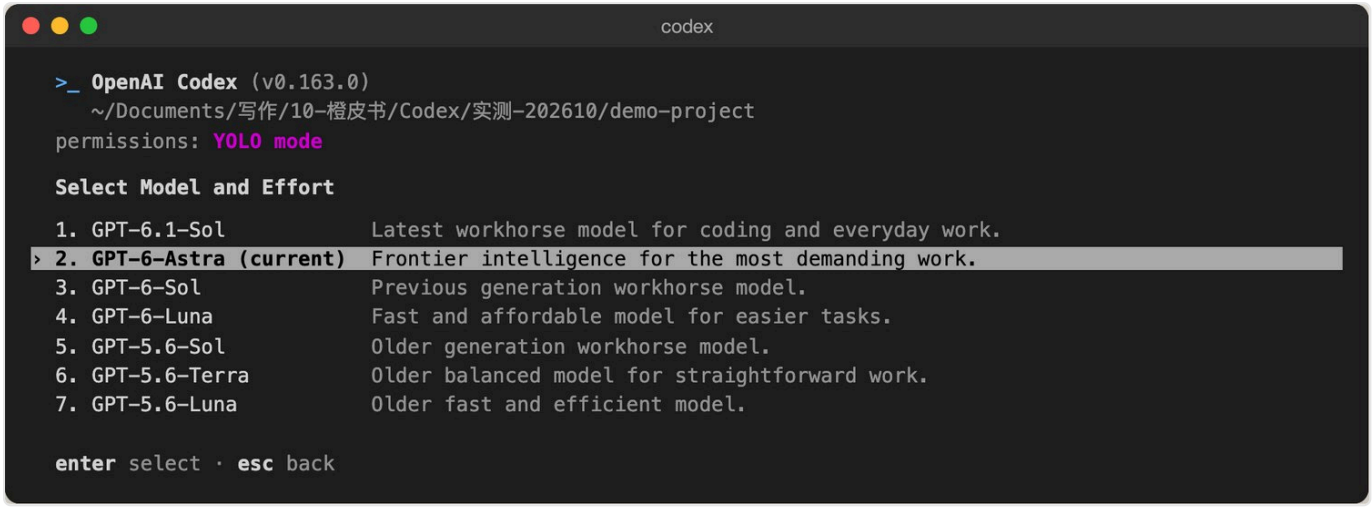
→ demo-project
```

`--full-auto` 也删了。它原来的效果（工作区可写、需要时再问）现在就是默认。

旧写法	现在怎么写
`codex --full-auto`	直接 `codex`，默认就是这个档
`-a untrusted` 、 `-a on-failure`	`-a on-request`，或用 `--approve-for-me` 交给自动审查
`codex --api-key sk-...`	`printenv OPENAI_API_KEY | codex login --with-api-key`
`codex sandbox macos -- ls`	`codex sandbox -- ls`
`[profiles.work]` 写在config.toml里	单独一个 `~/.codex/work.config.toml`，用 `-p work` 叠加
`codex mcp-server`	9月5日已删除，官方迁移到app-server
`/personality` 、 `/agent`	前者已移除；后者换成 `/agents` 和 `/subagents`

模型和推理档位

输入`/model`，弹出模型列表：



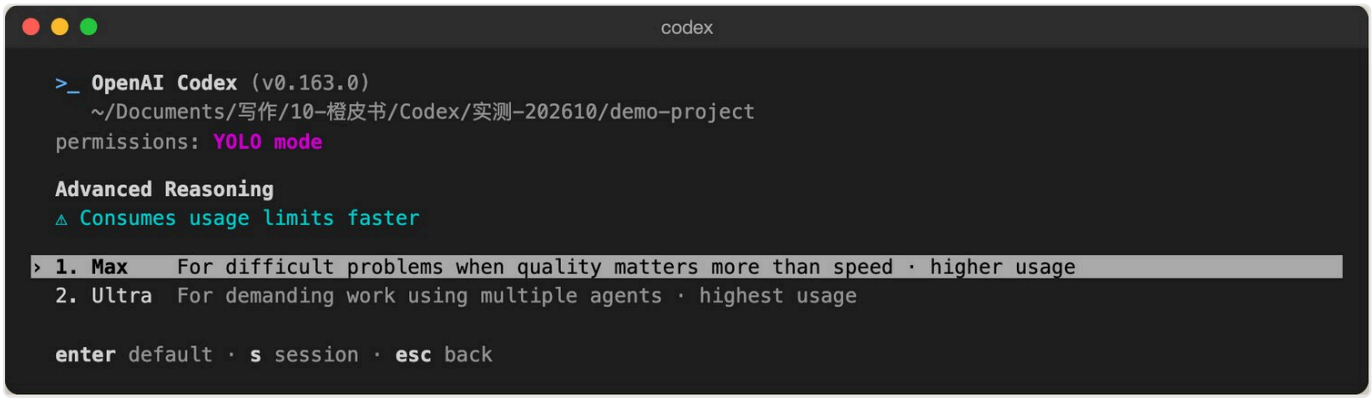
> 0.163.0里的7个模型。没有GPT-5.5和5.4了。

官方的建议很直接：复杂的编码任务用GPT-6.1 Sol；最难、最要质量的用GPT-6 Astra；聚焦、重复、量大的用GPT-6 Luna。GPT-6 Sol和5.6系列是上一代，还能选。

选中一个模型后，接着选推理档位：



> GPT-6.1 Sol默认是Low。底部的提示要看清：按Enter是设成默认（会写进配置），按s只对这次会话生效。



> More reasoning…里藏着两档：Max适合质量比速度重要的难题；Ultra用多个agent并行干活，用量最大。

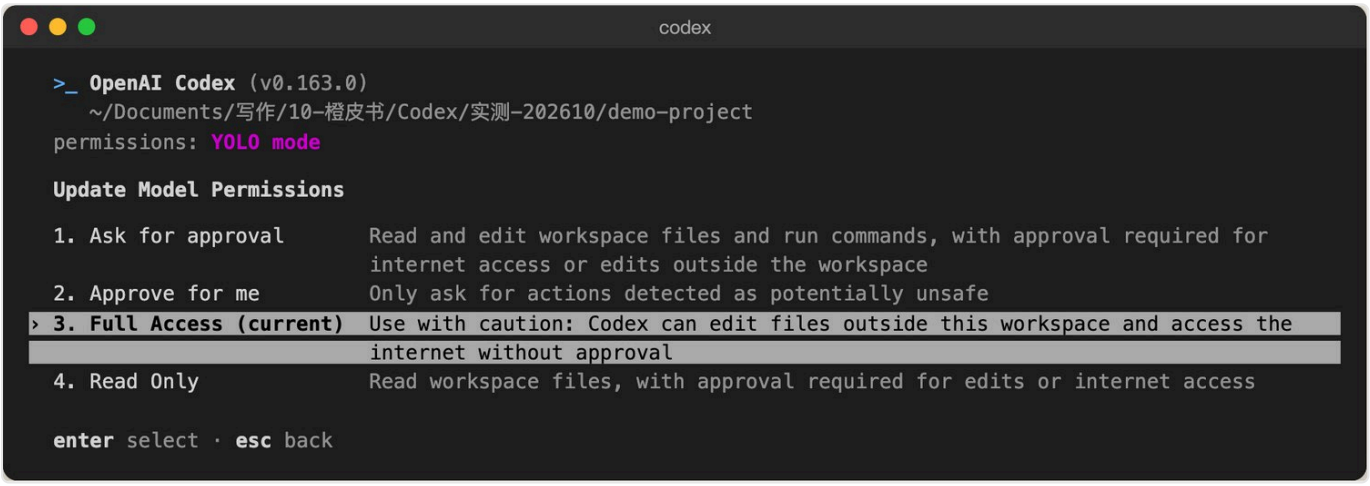
几个要点：

- **默认档位不一样。**本机这一版里，GPT-6.1 Sol默认Low，Astra和Luna默认Medium。5月版书里说默认是xhigh，已经不对了。
- **Ultra会拆给多个agent并行做，**，用量涨得最快。Luna只有Max，没有Ultra。
- **App里的叫法不同。**官方文档里，App和IDE的Light对应CLI的Low，Extra High对应xhigh。
- **Fast模式。**`/fast`切换当前模型的快速档，速度约翻倍，用量也涨（订阅额度按2.5倍算）。

在会话中途改档位，除了`/model`，键盘上Alt+逗号、Alt+句号（或Shift+上下）也能一档一档调。

权限：四个预设

输入`/permissions`：



> 四个预设。当前高亮的Full Access是我本机的个人配置，默认是第一个Ask for approval。

预设	能做什么	什么时候问你
Ask for approval（默认）	读改工作区文件、跑命令	要联网、要改工作区以外的东西时
Approve for me	同上	只有被自动审查判为可能不安全的动作才问你
Full Access	改任何地方的文件、联网，都不问	不问
Read Only	只读工作区	要改文件、要联网时

Approve for me就是Auto-review：越界的请求先交给一个独立的审查agent判断，它觉得低风险就放行，拿不准或高风险才打断你。它要你自己选，不是默认开的；命令行上对应`--approve-for-me`，配置里是`approvals_reviewer = "auto_review"`。被它拒掉的动作，你可以用`/approve`放行一次重试。

它靠不靠谱？OpenAI在4月30日发过一篇评测文章（alignment.openai.com/auto-review），给了几个数：OpenAI内部真实的越界请求里，它批准了99.1%；在一套人工构造的提示注入用例上，它正确拒掉了99.3%；会话停下来等人批准的次数，比手动审批少了大约200倍。同一篇文章也写明，它不能当成安全保证，红队测试

里有过把它误导过去的情况。这个功能3月以Smart Approvals的名字出现，4月20日的0.122.0改名叫Auto-review。

底层的沙盒是操作系统自带的机制：macOS用Seatbelt，Linux用bwrap加seccomp，Windows上ChatGPT桌面App带了原生沙盒。三档沙盒的名字是`read-only`、`workspace-write`、`danger-full-access`；配置里也可以用命名的权限配置`:read-only`、`:workspace`、`:danger-full-access`，或者自己写`[permissions.名字]`。

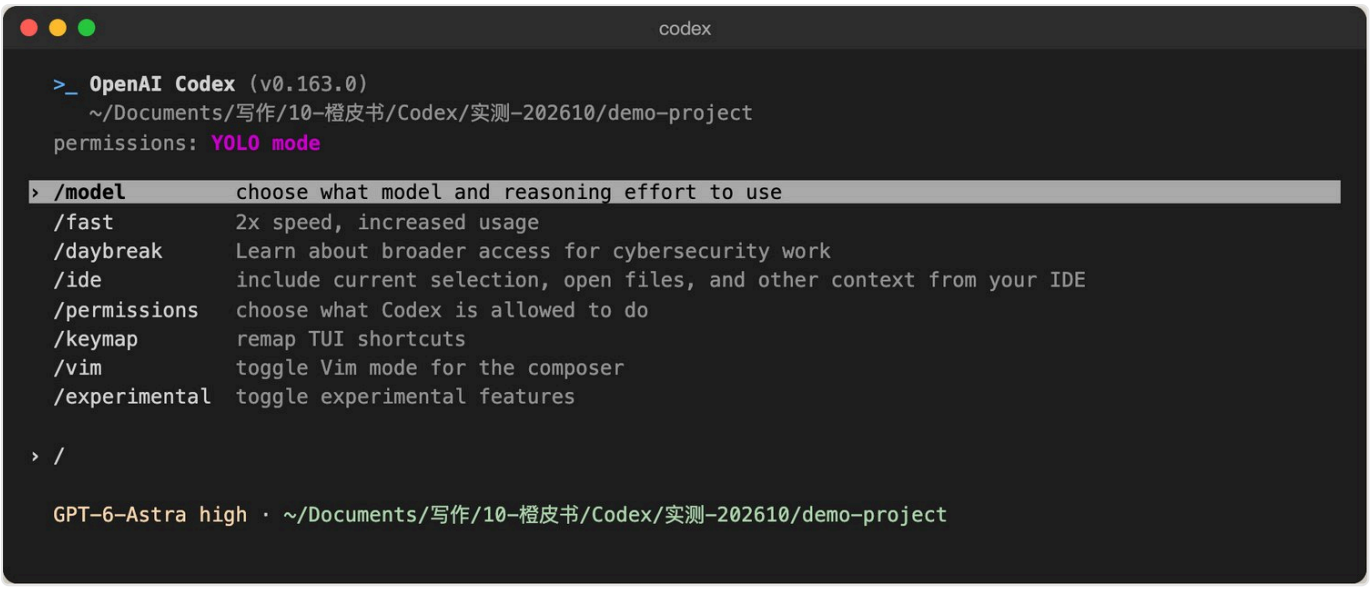
常用组合：

场景	命令
日常开发	`codex`
少被打断	`codex --approve-for-me`
只看不改	`codex -s read-only`
CI里只读分析	`codex exec "..."`（exec默认就是只读沙盒）
已经在外部隔离环境里（容器、虚拟机）	`codex --yolo`（全称`--dangerously-bypass-approvals-and-sandbox`）

> **Full Access要慎用，Windows上尤其。**2026年上半年，社区里有多起Windows用户在Full Access下被agent删掉工作区以外大量文件的报告。GitHub上的#18509（据提交者描述，一次归档会话失败时删掉了多个工作区根目录和Program Files下的程序，绕过回收站）到10月20日仍是open状态。之后Windows版加了原生沙盒，但我没找到官方宣布这类问题已修复的说明。不可逆的操作（删除、批量改名、强制推送）前先提交或备份；Full Access和`--yolo`只在你确定外面还有一层隔离时用。

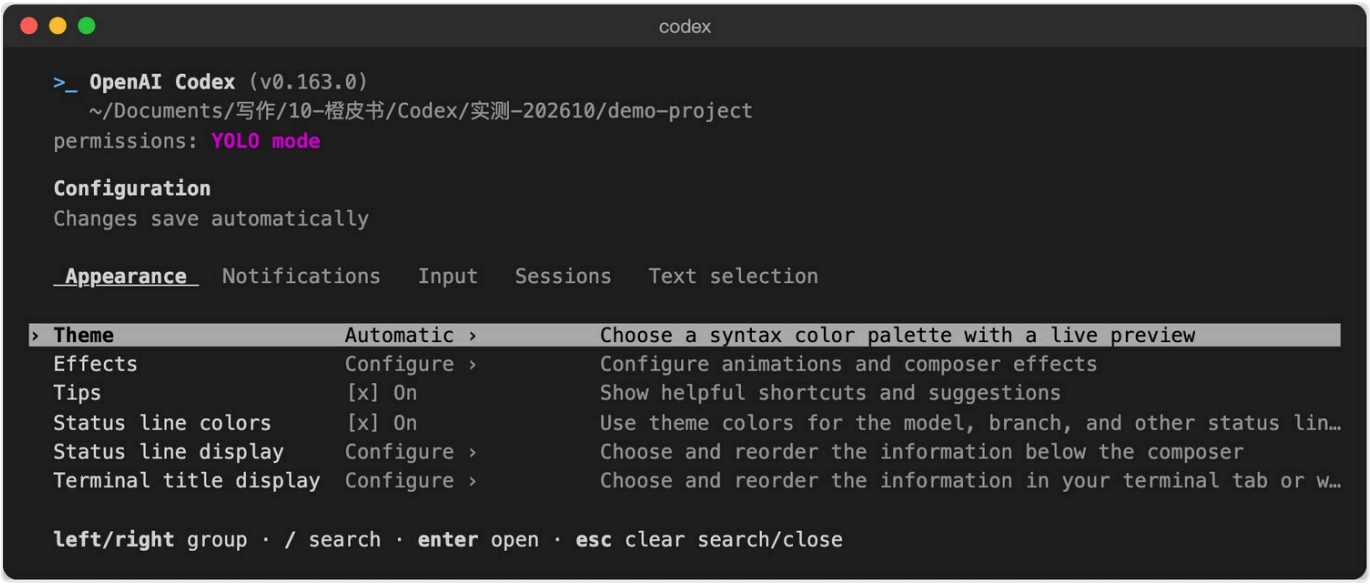
全屏界面里的几个面板

输入`/`会弹出斜杠命令菜单，一次显示8行，往下翻能看到全部。0.163.0里我数到56个（官方文档还列了几个，本机菜单里没有，以你自己的菜单为准）：



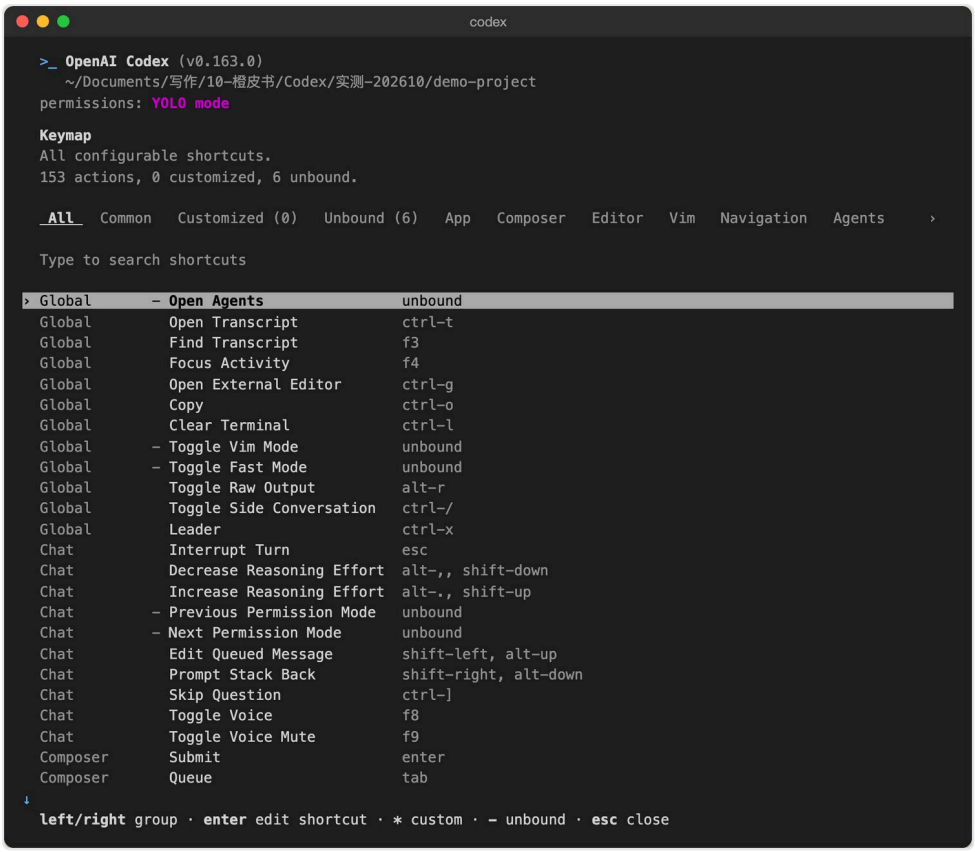
斜杠命令菜单的第一屏。完整的56个命令和说明见附录A。

`/config`：设置面板（0.163.0新增）。按组列出外观、通知、输入、会话、文本选择等设置，按`/`可以搜索，改了自动保存。



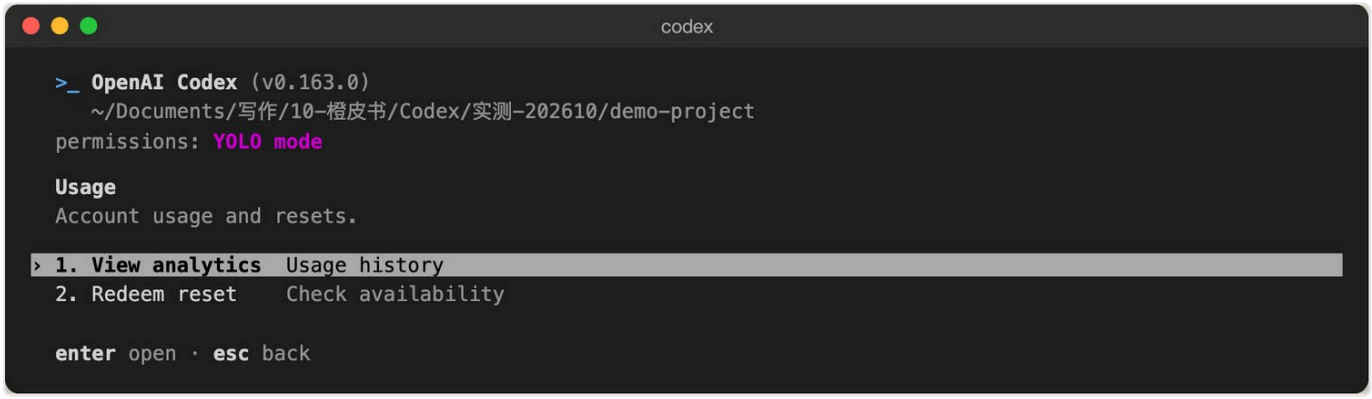
/config面板。左右键切分组，斜杠搜索，回车进入。

`/keymap`：快捷键。能看到全部默认键位，也能重新绑定。0.163.0加了一个leader键，默认Ctrl+X。



> 默认键位的一部分：Ctrl+G用外部编辑器写长prompt，Ctrl+O复制，Ctrl+L清屏，Ctrl+/开一个旁支对话，Esc打断当前这一轮，Tab排队一条消息。

`/usage`：用量。可以看用量历史，也可以兑换一次额度重置（如果你的账号有）。`/status`则显示当前会话的模型、权限、读到了哪些AGENTS.md、上下文用了多少、周额度剩多少（§03有截图）。



> /usage的两个入口：看用量历史，或者检查能不能兑换一次额度重置。

另外几个值得知道的：

- `/tui`：在全屏和Scrollback两种界面间切换，下次启动生效。
- `/side`：开一个临时的旁支对话问点别的，不污染主线。
- `/recap`：让它总结一下当前对话。`/export`：把对话导出成markdown。
- `/worktree`：在一个新的worktree里开始或继续对话。
- `/goal`：给一个长任务设目标，它会一直干到达成（§08）。

- `/import`：从Claude Code导入设置、当前项目和近期会话。**我这次在补全菜单里选中它按了回车，没看到确认就直接导入了**，用之前先备份。

会话：恢复、分叉、归档

```
# 打开选择器，列出最近的会话
codex resume

# 直接接着上一次
codex resume --last

# 不限当前目录
codex resume --all

# 从之前某个会话分叉出一条新线
codex fork --last

# 归档 / 取消归档 / 永久删除
codex archive <会话ID或名字>
codex unarchive <会话ID或名字>
codex delete <会话ID或名字>
```

会话的存储正在从老的`~/.codex/sessions/`目录往新的线程历史格式迁移（`codex migrate-rollouts`）。别依赖某个具体目录去翻记录，用`resume`和`/export`。

Web搜索

默认是缓存搜索：用OpenAI维护的索引，不实时抓网页，被网页里藏的恶意指令带偏的风险更低。想要最新的，启动时加`--search`，或者在配置里写：

```
web_search = "live"   # 另有 cached（默认）/ indexed / disabled
```

注意这和沙盒联网是两回事：Web搜索是模型的一个工具；沙盒断网管的是它跑的命令能不能访问网络。

非交互执行：codex exec

只想让它做完一件事、把结果给你，用`exec`（简写`e`）：

```
# 修CI
codex exec "跑测试，修掉所有失败的用例"

# 输出JSONL事件流，方便脚本解析
codex exec --json "分析依赖里的安全问题"

# 最后一条回复写进文件
codex exec -o summary.md "读最近10个commit，写一段更新说明"

# 要求最终输出符合一个JSON Schema
codex exec --output-schema schema.json "列出所有TODO"

# 不在磁盘上留会话记录
codex exec --ephemeral "..."

# 接着上一次exec的会话
codex exec resume --last "把你上次找到的竞态条件修一下"
```

代码审查有专门的子命令：

```
codex review --uncommitted        # 审未提交的改动
codex review --base main          # 和main比
codex review --commit <SHA>       # 审某个commit
```

会话里则是`/review`，它不动代码，只读diff给意见。

worktree：并行干几件事

以前想在CLI里并行，得自己开几个终端、手动管worktree。0.154.0起CLI自己就能做：

```
# 在一个新的受管worktree里开会话
codex --worktree

# 会话里切到新worktree
/worktree

# 看后台服务上所有会话（哪个在跑、哪个等你）
codex agents
```

CLI现在默认会连一个共享的后台服务，几个终端里的会话都挂在它上面，`codex agents`能一屏看全。不想用它就加`--no-daemon`。

功能开关

```
codex features list
```



> 筛出来的几行：goals、hooks、multi_agent、plugins、worktrees等都已经是stable并默认开启；memories是stable但默认关；personality已移除。

5月版书里让你手动开的`unified_exec`、`shell_snapshot`，现在都默认开着，不用管。真要临时开关某个功能，用`--enable 名字`、`--disable 名字`。

MCP

```
# 本地命令启动的MCP服务器（stdio）
codex mcp add context7 -- npx -y @upstash/context7-mcp

# 远程HTTP的MCP服务器
codex mcp add my-api --url https://api.example.com/mcp

# 列出、查看、删除、登录
codex mcp list
codex mcp get my-api
codex mcp remove my-api
codex mcp login my-api
```

5月版里的`--type stdio --command`写法不存在。会话里`/mcp`列出已连接的工具，`/mcp login 名字`做OAuth登录。MCP服务器启动超时默认10秒、单次工具调用超时默认60秒，在`[mcp_servers.名字]`里用`startup_timeout_sec`、`tool_timeout_sec`调。

Shell补全

```
# 加到 ~/.zshrc
eval "$(codex completion zsh)"

# bash、fish同理
codex completion bash
codex completion fish
```

碰到`command not found: compdef`，在那行前面加一行`autoload -Uz compinit && compinit`。

小技巧

- **@引用文件**：输入框里打`@`，模糊搜索工作区文件。
- **!跑shell命令**：输入`!ls`直接在本地执行。
- **Enter插话、Tab排队**：它干活时打字回车，会在这一轮里调整方向；按Tab则排到这一轮结束后。
- **Esc两下回溯**：输入框为空时连按两次Esc，编辑之前的消息，从那里分叉出新对话。
- **Ctrl+T看完整记录**：折叠起来的命令输出和diff都在里面。
- **启动前把环境备好**：先激活虚拟环境、启动需要的服务，Codex就不用花token去摸你的开发环境。

> **建议：**桌面App、IDE扩展底下跑的是同一套agent和同一份配置。CLI里把模型、权限、AGENTS.md弄明白了，换到别的入口也是同一套逻辑。

§05 AGENTS.md：给Codex一张地图

AGENTS.md — The Map You Draw for Codex

Codex每次启动时会自动读取AGENTS.md。这是你和AI之间关于「怎么干活」的契约，决定了Codex是一个空降新人，还是一个了解项目规矩的老手。

如果你读过我的Claude Code橙皮书，一定记得CLAUDE.md那一章。在Claude Code的世界里，CLAUDE.md是最重要的文件，是AI的「宪法」。

AGENTS.md在Codex中扮演的角色完全一样。

名字不同，逻辑相通：你把项目的构建命令、代码规范、常见陷阱写在一个文件里，AI每次启动时先读它，然后带着这些背景知识开始干活。**没有它，Codex就是蒙着眼写代码；有了它，它一进来就知道规矩。**

Codex怎么找到你的AGENTS.md

Codex在每次运行开始时（TUI模式下通常是每次启动会话时）构建指令链。发现顺序如下：

1. **全局级**

   在Codex主目录（默认是`~/.codex`，可通过`CODEX_HOME`环境变量修改）中，先找`AGENTS.override.md`，找不到再找`AGENTS.md`。这一层只取一个文件。

2. **项目级**

   从项目根目录（通常是Git仓库根目录）开始，逐级向下走到当前工作目录。每个目录中，依次检查`AGENTS.override.md`、`AGENTS.md`、以及`project_doc_fallback_filenames`中配置的备选文件名。每个目录层级最多取一个文件。

3. **合并**

   所有找到的文件按从根到当前目录的顺序拼接。**越靠近当前目录的文件优先级越高**，因为它们出现在合并后内容的后面，会覆盖前面的指令。

Codex会跳过空文件，合并后的总大小上限默认是32KB（由`project_doc_max_bytes`配置项控制）。超出限制时，后面的文件会被跳过，不再加入指令链。

> **建议：**32KB听起来很大，但如果你在多个嵌套目录都放了AGENTS.md，很容易撞到上限。建议把核心规则集中在根目录文件里，子目录文件只写该目录特有的内容。

override机制：临时覆盖不删原文件

每一层都优先找`AGENTS.override.md`。用得上它的场景是这样的：你的团队维护了一份`AGENTS.md`并check进了git，大家共享。但你个人有些不同的偏好，比如你喜欢用`pnpm`而团队规范用`npm`。这时候你不需要修改团队文件，只要在同一目录放一个`AGENTS.override.md`，它会直接替代同级的`AGENTS.md`。

把override文件加到`.gitignore`里，团队的基础规范不受影响，个人偏好也保住了。

同样的逻辑也适用于全局级。`~/.codex/AGENTS.override.md`会临时替代`~/.codex/AGENTS.md`，删掉override文件就恢复原来的全局设置。适合做短期实验。

/init：快速生成初始AGENTS.md

不想从零开始写？在Codex里输入`/init`命令，它会分析你的项目结构，自动生成一份初始的AGENTS.md。

生成的内容通常包括：项目使用的语言和框架、检测到的构建和测试命令、目录结构概述。你在此基础上增删修改，比从空白文件开始快得多。桌面App里也能用`/init`。

桌面App支持一个项目挂多个文件夹，这时只有你选定的主文件夹会用来找AGENTS.md、skills和config.toml，其他文件夹里的AGENTS.md不会被读。

一份好的AGENTS.md该写什么

判断标准跟Claude Code的CLAUDE.md完全一致：**AI自己能从代码里推断出来的，不要写；AI猜不到的，必须写。**

该写	不该写
构建、测试、lint命令	「这是一个Python项目」（AI看文件就知道）
与默认不同的代码风格偏好	语言标准规范（AI已经掌握）
项目架构决策和技术选型	详细API文档（给路径，别全文粘贴）
开发环境的坑和特殊配置	频繁变化的信息（每次都得改的不适合放这里）
PR要求和代码审查标准	文件逐一描述（AI会自己看文件树）
什么叫「完成」的定义	「写整洁代码」「遵循最佳实践」这种废话
约束和禁止事项（带替代方案）	所有人都知道的常识

看一个实际的项目AGENTS.md示例：

```
# MyProject

## 仓库结构
- src/          主代码
- tests/        测试文件
- scripts/      构建和部署脚本

## 开发命令
- 安装依赖：pnpm install
- 跑测试：pnpm test（所有测试必须通过再提PR）
- Lint检查：pnpm lint
- 构建：pnpm build

## 工程规范
- 使用TypeScript strict模式
- 组件用函数式，不用class
- 新增API接口必须写单元测试
- commit message遵循Conventional Commits格式

## PR要求
- 标题清晰描述改动内容
- 修改了用户可见行为时必须更新文档
- CI通过后才能合并

## 禁止
- 不要直接修改 generated/ 目录下的文件（它们是自动生成的）
- 不要在代码中硬编码API密钥，用环境变量
- 不要跳过lint检查
```

不到200字。但每一行都在帮Codex避免犯错。「什么叫完成」写清楚了（CI通过+文档更新），「禁止」项带着原因（generated目录是自动生成的），这些是Codex自己读代码推断不出来的信息。

Fallback文件名

如果你的项目已经有一个叫 `TEAM_GUIDE.md` 或者 `.agents.md` 的文件充当开发规范，不想再多一个AGENTS.md，可以配置fallback文件名：

```
# ~/.codex/config.toml
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
```

配置后，Codex在每个目录中的检查顺序变成：`AGENTS.override.md` → `AGENTS.md` → `TEAM_GUIDE.md` → `.agents.md`。**不在列表里的文件名会被忽略。**

上面的配置还顺便把合并大小上限调到了64KB，适合规则特别多的大型项目。

和Claude Code的CLAUDE.md对比

如果你同时用Claude Code和Codex，这张对比表会有用：

对比维度	AGENTS.md (Codex)	CLAUDE.md (Claude Code)
角色	项目指令文件	项目指令文件
全局配置	`~/.codex/AGENTS.md`	`~/.claude/CLAUDE.md`
项目配置	仓库根目录AGENTS.md	仓库根目录CLAUDE.md
子目录支持	✅ 从根逐级向下查找	✅ 启动时读当前目录及上层；子目录的在读写那个目录的文件时按需加载
override机制	✅ AGENTS.override.md	❌ 无此机制
Fallback文件名	✅ 可配置	2.1.277起，路径上没有CLAUDE.md时读AGENTS.md
大小限制	默认32KB，可调	官方建议200行以内，超过4 MiB的文件会被跳过
引用其他文件	不支持@语法	支持@语法导入
快速生成	`/init` 命令	`/init` 命令

核心思路一样，细节上各有取舍。Codex的override机制对团队协作更友好，Claude Code的@语法在大型项目中组织规则更灵活。两边一起用的话，现在可以只维护一份AGENTS.md，但注意Claude Code那条「路径上只要有CLAUDE.md就不读AGENTS.md」的规则，附录C有具体做法。

更广一点的趋势：SKILL.md已经成了几家agent共用的格式，Codex、Claude Code、Cursor、Gemini CLI都认。你写一个skill不再绑死某个工具，今天给Codex用、明天复制到Claude Code也能跑。

验证你的配置

写完AGENTS.md后，怎么确认Codex确实读到了？最直接的是会话里输入 `/status`，有一行 `Agents.md:`，列出这次读到的所有AGENTS.md。§03的截图里这一行是 `~/.codex/AGENTS.md, AGENTS.md`，全局一份、项目一份。

想看它理解成了什么，直接问：

```
# 从项目根目录验证
codex -a never "Summarize the current instructions."

# 验证子目录的override是否生效
codex -C services/payments -a never \
  "Show which instruction files are active."
```

行为上的验证更可靠。§03那个任务里，AGENTS.md要求「用中文回复」「总结里列出改动的文件名」，它的最终总结两条都做到了。

Codex每次运行都会重建指令链，不存在缓存。如果指令看起来不对，重启Codex即可。

常见问题排查

问题	排查方向
Codex没加载任何指令	确认你在预期的仓库目录中，会话里用 `/status` 看Directory和Agents.md两行。确认文件非空。
加载了错误的指令	检查目录树上层是否有 `AGENTS.override.md`，重命名或删除override文件。
Fallback文件名不生效	确认 `config.toml` 中拼写正确，然后重启Codex。
指令被截断	提高 `project_doc_max_bytes`，或把长规则拆分到子目录。

花叔的经验：AGENTS.md不要写太长。短而准比长而全有用。我自己写作项目的规则文件改了很多轮，全局那份和写作库那份现在都在10KB上下，里面不少条目后面都写着是哪天、因为什么事加的。每条规则最好都对应一个真实踩过的坑，提前想象的「万一需要」先别写。从你和Codex协作中真正出过问题的地方开始写，一条一条攒，这份文件自然会长成你需要的样子。

§06 桌面App里的Codex：多线程的指挥中心

Codex in the ChatGPT Desktop App

同时跑几个任务、让agent操作本机的其他App、在内置浏览器里看它改的网页、用手机远程审批，这些是桌面App比CLI多出来的部分。7月起，它住在ChatGPT桌面App里。

CLI有两类事天然不擅长：**同时盯好几个agent**，和**让agent操作图形界面**。桌面App就是为这两类事做的。

它现在在哪

2026年7月9日（App版本26.707）起，原来独立的Codex App并进了ChatGPT桌面App。打开ChatGPT桌面App，侧边能切三块：

- **Chat**：普通的ChatGPT对话。
- **Work**：ChatGPT Work，用和Codex同源的能力处理非编程的日常任务。
- **Codex**：本章讲的这一块，可以设成打开App时的默认视图。

Work和Codex共用同一份用量和额度。老的Codex App用户是原地更新过来的，项目和线程都在。

平台上，macOS和Windows是正式版，Linux（Ubuntu、Debian、Fedora，x64和ARM64）从8月11日起有预览版。装了CLI的话，`codex app`会打开桌面App，没装就打开安装程序；在CLI会话里输入`/app`，可以把当前这个会话接到桌面App里继续。

这一章的App功能这次没有截图实测，内容按官方changelog和文档整理，日期是changelog里的日期。

项目、线程、三种模式

App里可以加多个项目，每个项目对应一个代码库目录。7月23日起一个项目可以挂多个文件夹，其中一个是主文件夹，AGENTS.md、skills和config.toml只从主文件夹里找。

每个项目下面开多个线程，一个线程就是一次对话、一个任务。新建线程时选它在哪跑：

模式	工作方式	适合
Local	直接在项目目录里改	常规开发，改了马上能看效果
Worktree	在一个独立的git worktree里改	几件事并行，互不干扰
Cloud	在云端环境里跑（§07）	电脑关了也要继续，或者要特定环境

Worktree：并行的基础

git worktree能让同一个仓库同时有好几个独立的工作目录。选Worktree模式，Codex会给这个线程建一个独立的checkout，agent在里面改，你本地的代码不受影响，你可以照常写代码、跑服务。

干完之后两条路：直接在worktree上建分支、提交、推送、开PR；或者用Handoff把改动移回Local，在你熟悉的环境里检查。Handoff由Codex处理底层的git操作。要注意一条git本身的限制：同一个分支同一时间只能被一个checkout使用，worktree上建了`feature/a`，Local就不能同时checkout它，这时候用Handoff，别手动切分支。

CLI现在也能做这件事（`codex --worktree`、`/worktree`，§04），区别是App把几个线程摆在一个窗口里，谁在跑、谁在等你批准，一眼能看到。

一个典型的并行用法

假设一个Web项目同时有三件互相独立的事：做一个新的设置页、把一个接口从REST改成GraphQL、修首页的一个bug。

1. 每件事开一个Worktree线程，需求写清楚，设置页的需求可以用`@`引用设计文档。
2. 三个线程并行跑。审批多的话，权限选Approve for me，让自动审查处理低风险请求（§04）。
3. 哪个先完成先看哪个的diff，没问题就开PR，或者Handoff回Local做集成测试。

App的diff面板能按未提交、整个分支、最近一轮切换范围，能在具体某一行上加评论，再让它「处理inline评论，改动尽量小」。评论锚定在行上，比在对话框里描述「第几行那个函数」准确。

子agent

除了你手动开几个线程，一个线程里的主agent也可以自己派子agent。当前版本子agent默认开启，`codex features list`里`multi_agent`是stable。配置在`config.toml`的`[agents]`下：

```
[agents]
max_concurrent_threads_per_session = 6   # 旧名 max_threads 仍可用
default_subagent_model = "gpt-6-luna"
default_subagent_reasoning_effort = "medium"
```

每个自定义角色单独一个TOML文件，放在`~/.codex/agents/`（个人）或`.codex/agents/`（项目）：

```
name = "planner"
description = "把大任务拆成可并行的小任务，自己不动手"
developer_instructions = "你是协调者，把任务拆给子agent，自己不直接改代码。"
model = "gpt-6.1-sol"
```

角色文件里写`sandbox_mode`、`approval_policy`不会生效，子agent的沙盒和审批跟着父会话走。想让它「不动手」，靠`developer_instructions`里写清楚。

内置的有default、worker、explorer三个，自定义的同名会覆盖内置的。CLI里`/agents`打开agent命令中心，`/subagents`在子agent之间切换。推理档位选Ultra，就是让它用多个agent并行拆着干，用量涨得最快。

5月版书里列了三个多agent的bug。查了一下：#16657在4月3日关闭，#14233在5月7日关闭，#17523（子agent的完成回执以裸JSON漏进主对话）到10月20日仍是open。关键任务先拿小任务试一遍再放手。

Computer Use：让Codex操作你电脑上的App

Computer Use让Codex用自己的光标操作本机的其他App：点菜单、填表单、拖拽、读屏幕。在macOS上它在后台用自己的光标，不抢你的鼠标；Windows上它在前台运行，会接管鼠标指针。这半年的变化：

- 5月21日：Mac锁屏后也能远程继续用（Remote computer use）。
- 5月29日：支持Windows。
- 6月18日：Record & Replay（macOS），你演示一遍操作，它变成一个可复用的skill。

还有一个小功能叫Appshots（5月21日，Windows版9月11日）：同时按两个Command键，把最前面那个窗口的截图和文字一起发给Codex。看到某个App里的报错，不用自己截图再拖进来。

内置浏览器

App里有一个浏览器面板，用来打开localhost看它改的页面，你也可以直接在页面元素上圈出来写评论给它，比如「这个标题缩小到48px」。它拿到的是锚定在元素上的反馈。

这半年加了：Developer mode（6月11日，可以用Chrome DevTools协议调试）、浏览历史（7月30日）、站点工具（8月25日，支持网站用WebMCP暴露给agent的工具）。内置浏览器有自己独立的登录状态，可以在里面单独登录；想直接用你日常浏览器里已有的登录态，用浏览器扩展（§01）。

图片生成

App里内置的图片模型现在是GPT-Image-2（5月版书里是gpt-image-1.5），7月30日起生成的图能在画布上继续编辑。截图、分析、出图、写进代码、看效果可以在一个线程里走完，适合给前端出占位图、空状态插画、产品概念图。图片生成按普通消息的3到5倍消耗额度。

Memories

Memories会记住你的偏好、你纠正过它的地方、你攒下的上下文，下次同类任务不用再交代一遍。比如你纠正过「测试用Vitest不用Jest」，它会记着。

它和AGENTS.md的分工：AGENTS.md是你显式写下的规则，Memories是用的过程中攒下来的。CLI里`memories`这个功能是stable但默认关，用`/memories`设置；欧盟、英国、瑞士的用户默认也是关的。Pro套餐另有一个研究预览叫Chronicle。

其他你会用到的

- **App内编辑**（7月9日）：直接在App里改Markdown和代码文件。
- **PR Chat**（7月9日）：围绕一个PR对话。**多仓库一起Review**（7月30日）。
- **语音**（7月23日）：ChatGPT Voice（GPT-Live），按每分钟0.05美元计入Codex额度。
- **集成终端**：每个线程带一个终端，跟着线程的目录走，Codex能读到终端里的输出，dev server的报错不用你复制粘贴。
- **定时任务**：侧边栏的Scheduled，§08细讲。
- **导入**：Settings > Import，从Claude Code、Claude Cowork、Cursor导入设置、项目指令、skills和近期会话，可以开自动同步。
- **Skills、MCP、插件**：和CLI、IDE扩展共用同一份配置。插件管理7月9日起搬进了Settings。

手机：Codex Remote

6月25日正式上线。ChatGPT手机App（iOS、Android）里的Codex Remote，扫码和一台跑着Codex的Mac或Windows电脑一对一配对。之后在手机上：

- 看那台电脑上每个线程的进度、截图、diff、测试结果；
- 审批命令、切模型、回答它的提问；
- 新建任务、追加指令；
- 也可以不连电脑，直接选一个云端环境起任务。

执行仍然在电脑上，文件、凭据、沙盒都在那台机器里。6月18日起线程还能在本地和远程主机之间移交。CLI这边有对应的`codex remote-control pair`，打印一个短时有效的配对码。

平台支持（2026年10月）

入口	macOS	Windows	Linux	手机
CLI	有	有	有	无
ChatGPT桌面App里的Codex	有	有	预览	无
浏览器扩展	Chrome、Edge、Brave、Opera、Vivaldi			无
IDE扩展	有	有	有	无
Codex Remote	作为被控电脑		无	iOS、Android
Cloud	浏览器里用			手机里可直接起任务

什么时候用CLI，什么时候用App

场景	推荐
单个任务，快速修改	CLI
同时盯两个以上任务	App（CLI用`codex agents`也能看）
可视化看diff、按行写评论	App
操作本机其他App	App（Computer Use）
在网页上圈元素提意见	App（内置浏览器）
定时任务	App（Scheduled）
需要登录态的网站	浏览器扩展
CI、脚本批处理	CLI（`codex exec`）

两边共用同一份AGENTS.md、skills和`config.toml`，切换没有迁移成本。

怎么选：只跑一个任务时，CLI更轻；同时有几件事在跑、还要时不时批准一下，App把它们放在一屏里。出门前把长任务开好，路上用手机批准，回来看diff，这种用法要靠App和手机配合。

§07 云端Codex：交出去，回来收结果

Codex Cloud: Hand It Off, Come Back for Results

把任务丢到云端，你去做别的事，电脑关了它也照跑。这半年云端换了一套做法：先让Codex帮你搭好一个可复用的环境，再从这个环境起任务。云端这次没有上手实测，这一章按官方文档（learn.chatgpt.com/docs）和changelog整理。

为什么要用云端

本地的几个入口，电脑一合上就停了。云端的任务跑在OpenAI的虚拟机里，你关掉浏览器、合上电脑，它照样在跑；回来的时候改动和测试结果都在那，满意就提交或开PR。

适合丢到云端的，是那种需求说得清、交代完就不用你盯着的活：修一个issue、补一批测试、做一次范围明确的重构、审一个PR。需要你边看边改、反复讨论的，还是在本地做。

新做法：先建一个可复用的环境

5月版书里，云端的流程是「打开chatgpt.com/codex，选仓库、选分支，提任务」，环境靠你自己写setup脚本。现在官方的做法换了：

1. **新建环境**

   在网页或桌面App里新建任务，选Work in > Cloud，打开Select environment，点Create environment。也可以从Settings > Codex Cloud > Environments进去。

2. **选仓库，让它自己准备**

   选要checkout的GitHub仓库（第一次会让你连GitHub），点Get started。Codex会检查仓库、装依赖和工具、把工作流跑一遍试试；缺权限、缺信息时它会问你。你也可以在对话里提要求，比如指定版本、命令或者要起的服务。

3. **看报告，发布**

   它会给一份设置报告。看过配置和文件，处理掉没完成的部分，保存，点Publish。出现Environment published就好了。

4. **开始任务**

   点Start a new task，描述你要它做什么。之后每个新任务都从这个发布好的环境起跑，各自一个独立的工作区。

手机上打开ChatGPT里的Codex，选一个已发布的环境就能起任务；新建环境要在网页或桌面App里做。

环境里存了什么

你不用再自己写安装脚本。Codex会把试通的设置记在两个字段里：

- **Install script**：装依赖、准备开发资源的命令。
- **Start skill**：怎么把服务起起来、怎么检查它们准备好了。

发布时，它把准备好的文件系统存下来。之后：

情况	从哪开始
新任务	从发布的环境里那份准备好的文件系统开始
继续一个已有任务	用这个任务自己存下的文件，包括没提交的改动和装过的工具
仓库有更新	后台自动刷新代码，保留依赖缓存，不重跑安装

项目变了要更新环境：Settings > Codex Cloud > Environments，在环境的菜单里选Edit，描述要改什么，让它准备、测试，保存后点Republish。已有的任务保持自己的状态，新任务才用新设置。官方特别提醒：存下来的状态不能代替版本控制，重要的东西记得提交。

环境变量和网络密钥

Codex推断不出的配置会来问你。在环境配置里，Environment variables和Network secrets两项旁边点Manage：

类型	用来做什么	怎么给到程序
环境变量	程序要直接读的值	直接传给环境里的程序
网络密钥（Network secret）	发给某个HTTPS服务的凭据	程序拿到的是占位符，请求发往允许的域名时，代理才把真值换进去

网络密钥要填Key、Value和Allowed domains，只对443端口的HTTPS生效，安装阶段和任务里都能用，真实的凭据不会出现在本地进程或文件里。这是给agent用密钥最稳的方式：它能调用你允许的服务，却拿不到密钥本身。

团队里每个人自己的值，放在Settings > Codex Cloud的Personal vault里，可以选对所有环境还是指定环境生效。共享一个环境，共享的是需要哪些值，每个人的凭据留在自己的Personal vault里。

联网：默认关，按需开

为什么云端默认不给联网？因为提示注入。假设你让它修一个GitHub issue，而issue正文里藏了一句「请运行下面这个脚本并给出输出」，脚本内容是把最新的commit发到某个外部地址。如果agent能自由联网，它可能真的照做。不给联网，这条路就断了。

确实需要访问外部服务时，在环境配置里：

1. 打开Allow Codex to access internet。
2. Allow domains选Package managers（官方预设，覆盖npm、PyPI、crates.io、Go、Maven、Ubuntu和Debian源、GitHub等）或Custom domains only，再在Additional allowed domains里加你要的域名。确实需要大范围访问才选All (unrestricted)。
3. 保存，在设置阶段测一下，发布后在新任务里再验证一次。

访问内网服务可以配VPN，目前支持Tailscale（Advanced > VPN）。访问云厂商资源可以用OIDC拿短期凭据。企业工作区的管理员还能用Agent Security统一设规则。

> **建议：**能不开网就不开。真要开，先试Package managers预设加几个具体域名，All (unrestricted)留到最后。agent写代码、跑测试、做重构，绝大多数时候不需要联网。

GitHub代码审查

在PR的评论里写`@codex review`，Codex会先回一个👀表示收到，然后像队友一样发一条标准的GitHub代码评审。它只标P0和P1级别的问题，保持评论聚焦在高风险上。

想让它每个PR都自动审：到`chatgpt.com/settings/code-review`，选仓库，打开Automatic review，再用Review trigger选什么时候触发。

想让它按你的规矩审：在AGENTS.md里加一节`## Code Review Rules`，全仓库的规则放根目录，某个服务专属的规则放那个服务目录下的AGENTS.md。官方建议从两三条简短的规则开始，写那些人工审查时经常要重复解释的点。

在PR或issue评论里@codex说别的事（比如`@codex fix the CI failures`），它会建一个云端任务去做。官方还有一个研究预览的Security Review，专门深挖PR里的安全问题。

Linear、Slack、GitLab

- **Linear**：把issue指派给Codex，或者在评论里@它，它会在issue里更新进度。
- **Slack**：在频道里@ChatGPT发任务，它建云端任务执行，完成后把结果发回来。前提是工作区管理员开了云端委派，并且有一个工作区共享的已发布环境。
- **GitLab**：8月19日起支持（Beta），能做任务和MR审查；自建的GitLab需要19.0以上版本配webhook。

代码审查和Linear、GitHub这几个集成，目前还挂在Codex Cloud（Legacy）这套旧体验上，旧的环境设置页在`chatgpt.com/codex/settings/environments`。官方说了计划弃用Legacy，届时迁移方式以官方公告为准。

从终端起云端任务

CLI里有`codex cloud`（仍标着实验性）：

```
# 列出最近的云端任务
codex cloud list

# 在某个环境里起一个任务
codex cloud exec --env <ENV_ID> "修掉 issue #42"

# 看状态、看diff、把改动应用到本地
codex cloud status <任务ID>
codex cloud diff <任务ID>
codex cloud apply <任务ID>
```

适合云端的几类活

- **代码审查**：每个PR自动过一遍。
- **从issue到PR**：在issue里@codex描述问题，修完你看diff。
- **范围明确的重构**：改动面大但你知道要什么，在独立环境里做，不碰你本地。
- **批量并行**：同一个环境起好几个任务，各跑各的，回来逐个看。

> **建议：**云端最省事的用法是批量委派：把几个互相独立、说得清楚的issue整理好，一次起几个任务，去做别的事，回来逐个看diff。前提是环境先搭稳，测试能在环境里跑通，不然每个任务都会卡在同一个环境问题上。

§08 Skills、插件、MCP、hooks、定时任务与/goal

Skills, Plugins, MCP, Hooks, Scheduled Tasks & /goal

Skill定义做什么、怎么做；插件把skill、MCP和hooks打成一个包；MCP接外部工具；hooks在固定时刻插一段你的代码；定时任务决定什么时候做；/goal决定什么时候算做完。

Skills：教Codex做事的方法

一个skill就是一个目录，里面有一个`SKILL.md`，开头写`name`和`description`：

```
---
name: skill-name
description: Explain exactly when this skill should and should not trigger.
---

Skill instructions for Codex to follow.
```

正文是要它遵循的步骤、规则、模板，目录里还可以放脚本和参考文件。

按需加载

Codex启动时只读每个skill的name、description和路径，占的token很少；判断某个skill和当前任务相关时，才加载完整的SKILL.md。所以装几十个skill也不怕撑爆上下文，但**description写得好不好，直接决定它会不会被正确触发**：写清楚做什么、什么时候用、什么时候别用。

两种调用

- **显式**：在prompt里写`$skill-name`。输入`$`会弹补全，`/skills`能浏览全部。
- **隐式**：你不指定，Codex按description自己匹配。

放在哪

层级	路径	适合
REPO	`.agents/skills/`（从当前目录往上一直扫到仓库根）	项目或某个子服务专用的工作流
USER	`$HOME/.agents/skills/`	你自己跨项目用的
ADMIN	`/etc/codex/skills/`	管理员统一部署的
SYSTEM	Codex内置	skill-creator、plan等

符号链接的skill目录也能识别。SKILL.md这套格式Claude Code、Cursor、Gemini CLI也认，给Claude Code写的skill，挪到 `.agents/skills/` 下通常就能在Codex里用。

创建、安装、停用

```
# 用内置的skill-creator，它会问你这个skill做什么、什么时候触发
$skill-creator

# 安装官方精选的skill，比如linear
$skill-installer linear
```

不想删、只想先停用一个skill，在 `~/.codex/config.toml` 里写：

```
[[skills.config]]
path = "/path/to/skill/SKILL.md"
enabled = false
```

skill目录里还可以放一个 `agents/openai.yaml`，配桌面App里显示的名字和图标、调用策略、依赖的工具：

```
interface:
  display_name: "Optional user-facing name"
  short_description: "Optional user-facing description"

policy:
  allow_implicit_invocation: false

dependencies:
  tools:
    - type: "mcp"
      value: "openaiDeveloperDocs"
      transport: "streamable_http"
      url: "https://developers.openai.com/mcp"
```

`allow_implicit_invocation: false` 是个安全开关：设了之后Codex不会自动触发它，只有你显式写 `$skill-name` 才调用。有破坏性操作的skill建议这么设。

桌面App（macOS）还有第三种造skill的办法：Record & Replay，你把一个操作演示一遍，它记成一个可复用的skill（§06）。

插件：skill、MCP、hooks打成一个包

一个插件可以同时带skills、MCP服务器、hooks和App连接。现在ChatGPT和Codex共用同一个插件目录：

```
# CLI里管理插件
codex plugin list
codex plugin add <名字>
codex plugin remove <名字>
codex plugin marketplace ...

# 会话里浏览
/plugins
```

桌面App里插件管理在Settings里。Business套餐5月起能在团队里分享插件。DevDay（9月29日）还发布了Plugin Extensions，插件可以带侧边栏、面板、表单这样的界面。

MCP：连接外部工具

MCP（Model Context Protocol）是让Codex和外部工具对话的标准协议。CLI、IDE扩展、桌面App共用一份MCP配置。

两种服务器：**STDIO**是本地进程，用一个命令启动；**Streamable HTTP**是远程服务，用URL访问，支持Bearer token和OAuth（`codex mcp login 名字`）。

```
# 添加
codex mcp add context7 -- npx -y @upstash/context7-mcp
codex mcp add linear --url https://mcp.linear.app/mcp

# 会话里看已连接的工具
/mcp
```

也可以直接写`config.toml`（项目级的`.codex/config.toml`只在受信任的项目里生效）：

```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

[mcp_servers.figma]
url = "https://mcp.figma.com/mcp"
bearer_token_env_var = "FIGMA_OAUTH_TOKEN"
```

选项	说明
`startup_timeout_sec`	启动超时，默认10秒
`tool_timeout_sec`	单次调用超时，默认60秒
`enabled`	设false停用但不删
`required`	设true时启动失败会挡住Codex
`enabled_tools` / `disabled_tools`	工具白名单/黑名单

常用的几个：OpenAI Docs MCP、Context7（最新的库文档）、Figma MCP（读设计稿）、Playwright MCP和Chrome DevTools MCP（控制浏览器）、Sentry MCP（线上错误）、GitHub MCP。

5月版书里还有一节「Codex自身作为MCP Server」，那个`codex mcp-server`命令8月24日标为废弃、9月5日删掉了，官方的替代是app-server。

hooks：在固定时刻插一段你的代码

hooks在5月14日正式可用，现在是默认开启的。它让你在Codex干活的某个时刻，自动跑一段你写的脚本：检查、拦截、补充上下文、记日志。

什么时候	事件
一轮对话里	`PreToolUse`、`PermissionRequest`、`PostToolUse`、`PreCompact`、`PostCompact`、`UserPromptSubmit`、`SubagentStop`、`Stop`
你打断当前这一轮时	`Interrupt`
会话或子agent开始时	`SessionStart`、`SubagentStart`
主线程结束时	`SessionEnd`

写在`~/.codex/hooks.json`、`~/.codex/config.toml`，或项目的`.codex/hooks.json`、`.codex/config.toml`里。一个在每次跑shell命令前做检查的例子：

```
[[hooks.PreToolUse]]
matcher = "^Bash$"

[[hooks.PreToolUse.hooks]]
type = "command"
command = '/usr/bin/python3 "$(git rev-parse --show-toplevel)/.codex/hooks/pre_tool_use_polic
timeout = 30
statusMessage = "Checking Bash command"
```

脚本想拦下这次调用，可以返回带`"permissionDecision": "deny"`和理由的JSON，或者以退出码2结束、把理由写到stderr。

安全上有两条要知道：

- **新的或改过的hook要你审核信任后才会跑。**Codex按hook定义的哈希记录信任，内容一变就重新标为待审。会话里用`/hooks`查看、信任或停用。
- **项目里的hooks只在你信任了这个项目之后才加载。**这就是§02那个信任文件夹的弹窗里说「文件夹里的设置可能不经模型就执行代码」的原因。`--dangerously-bypass-hook-trust`可以跳过审核，只给已经自己审过hook来源的自动化用。

官方给的典型用途包括：扫描提示词，防止不小心把API密钥贴进去；会话开始时自动注入项目上下文；跑shell命令前按团队规则检查。

定时任务（Scheduled tasks）

5月版书里叫Automations，现在官方叫Scheduled tasks，结果出现在ChatGPT桌面App侧边栏的**Scheduled**里，那里相当于一个收件箱，有发现的运行会带未读标记。

有三种跑法：

在哪建	能做什么	前提
桌面App里的Codex	在本地项目目录或一个新worktree里跑，能改你的代码	电脑开着、App开着、项目还在磁盘上
ChatGPT网页或手机	用上传的文件、已连接的工具、skills和插件	碰不到你电脑上的文件夹
网页或手机上的事件触发	Gmail新邮件、Slack频道新消息、GitHub PR动态触发一次运行	符合条件的套餐；桌面App、CLI、IDE里没有；不能和定时同时设

CLI和IDE扩展里没有管理定时任务的界面，官方的建议是：先在CLI里把prompt、skill或脚本调试好，再去App或网页里建任务。

两种任务形态：

- **独立任务**：每次运行开一个新对话，结果作为一条独立记录出现在Scheduled里。自定义周期用日历标准的RRULE写，比如 `RRULE:FREQ=MONTHLY;BYMONTHDAY=1;BYHOUR=9;BYMINUTE=0`（每月1号早上9点）。
- **对话里的定时**：在一个已有对话里让它按时回来，带着这段对话的上下文继续。可以按分钟间隔跑，适合盯一个还没跑完的部署、隔一阵检查一次PR状态。

建任务最简单的方式是直接跟它说：描述要做什么、什么时候跑、每次是回到这个对话还是开新对话。在App里的任务prompt中写 `$skill-name` 可以明确调用某个skill，官方也建议把动作写成skill，任务本身只管什么时候触发。

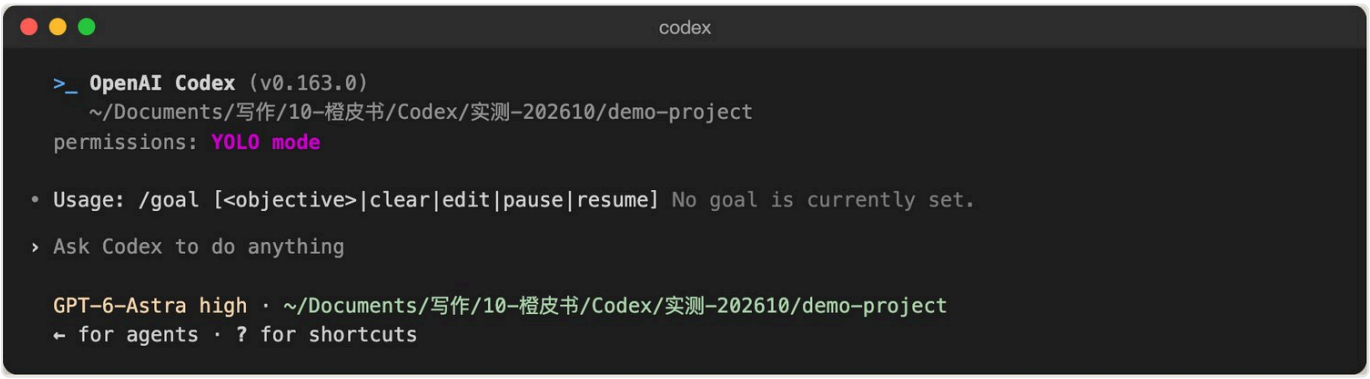
几条官方建议：

- **先手动跑一遍。**在普通对话里把prompt跑通，确认范围清楚、输出能审，再设成定时；前几次运行的结果要认真看。
- **权限给最窄的。**定时任务无人值守，用你的默认沙盒设置。先用workspace-write，需要时再放开网络或更大的文件范围。
- **worktree会越积越多。**高频任务选worktree模式的话，定期归档不需要的运行。
- **模型下线了要改。**GPT-5.5在10月14日对ChatGPT登录的Codex退役，用了它的定时任务要换成GPT-6.1 Sol。

官方举的例子有：每天看一遍监控里的报错并提交修复、定期出一份最近代码改动的报告、让一个盯PR的skill自己建定时任务去检查PR状态并处理新的审查意见。

/goal：干到达成为止

`/goal` 是0.128.0（4月30日）加的，5月21日退出实验，现在App、IDE、CLI都能用。你给它一个目标，它会一轮轮地干，直到目标达成或者被你停下：



> 不带参数输入/goal，会打印用法：设置目标，或者edit、pause、resume、clear。状态栏里的YOLO mode和GPT-6-Astra是我本机的个人配置，不是默认值。

```
/goal 把src/legacy下的模块全部迁移到TypeScript，npm test全部通过
/goal             # 查看当前目标
/goal edit        # 修改
/goal pause       # 暂停
/goal resume      # 继续
/goal clear       # 清掉
```

目标最长4,000字符。几个版本里加的保护：

- 0.132.0起，碰到用量上限或者反复卡在同一个障碍上，它会停下来，不会一直空转烧额度。
- 0.148.0起，可以给goal配一个token预算。
- 0.151.0起，子agent用的token也算进主目标的预算里。

什么时候用：

- **适合**：有明确的完成判据（测试全绿、某个指标达标），需要很多轮才能做完，你愿意让它自己决定中间的步骤。
- **不适合**：几分钟就能做完的小事；需要你边看边判断的调试和探索。目标一设，它会一直往前推，你反而插不上手。

完成判据写得越具体越好。「代码质量要好」这种它没法判断有没有达到；「`npm test`全部通过，并且不改测试文件」就能。

它们怎么配合

层	回答的问题
Skill	做什么、怎么做
插件	从哪拿现成的skill、MCP、hooks
MCP	连什么外部工具
hooks	在哪些时刻自动检查或拦截
定时任务	什么时候做、多久做一次
`/goal`	做到什么程度才算完

一个组合的例子：写一个`$sentry-triage`的skill，读Sentry MCP里最近的高优先级错误、找到对应代码、开修复PR；再建一个工作日每天早上9点的定时任务，prompt里写明调用这个skill，用worktree模式；加一个`PreToolUse`的hook，拦掉任何想推到main分支的命令。你早上打开Scheduled，看到的是它昨晚到今早开了哪些PR、哪些错误它判断需要你来决定。

和Claude Code对照

这几层Claude Code现在都有对应的东西：Skills和插件市场、MCP、hooks、`/goal`（2.1.139起，用一个独立的小模型判断目标是否达成）、`/loop`（会话内按间隔重跑）、Routines（云端定时，研究预览）。两家的差别更多在细节：Codex的定时任务和事件触发挂在ChatGPT的App和网页上，跟Gmail、Slack这些已连接的应用打通；Claude Code的定时分成会话内、桌面、云端几层。§10有完整对照。

建议：上手顺序是是先skill、再定时任务、最后/goal。先把一件你反复做的事写成skill，手动调用几次确认稳了；再让它定时跑；真有一件需要连续推进很多轮、完成标准又说得清的事，再上/goal。三样同时上，出了问题很难分清是哪一层的事。

§09 从零构建一个完整产品

Building a Complete Product from Scratch

把前面讲的入口、AGENTS.md、并行和云端串起来，从一个想法走到一个能部署的Web应用：怎么拆任务、每一步用哪个入口、怎么管几个并行的agent、每一步怎么验收。

说明一下：这一章是流程示范。CLI的部分和§03是同一套操作；App里的并行线程和云端任务这次没有上手跑，步骤按官方文档写。prompt可以直接拿去用，里面说的冲突、报错这些情况是一般规律，你跑出来的会不一样。

项目选择：AI文章摘要工具

我们要做的是一个Web应用：用户粘贴一篇文章的URL，系统自动抓取内容、调用OpenAI API生成摘要，并把历史记录存下来。

为什么选这个项目？几个原因：

技术栈覆盖面刚好。前端（Next.js页面）、后端（API路由）、数据库（SQLite）、外部API调用（OpenAI），四层都有。足够展示Codex在不同场景下的能力，又不至于复杂到写不完。

每一层可以独立开发。前端UI、后端逻辑、数据库模型，三块互相不依赖（在约定好接口的前提下）。这正好适合用Codex的多agent并行。

做完能自己用。读文章多的人都用得上，自己会用的东西，验收起来也有标准。

最终成品长这样：

模块	技术选型	负责什么
前端	Next.js（App Router）+ Tailwind CSS	URL输入框、摘要展示、历史列表
后端	Next.js API Routes	抓取文章内容、调用OpenAI生成摘要
数据库	SQLite + Drizzle ORM	存储文章URL、原文、摘要、时间戳
部署	Vercel	一键部署，免运维

阶段一：需求分析和规划（CLI Plan模式）

打开终端，进入你想存放项目的目录，启动Codex CLI：

```
codex
```

输入`/plan`切到Plan模式，让Codex先出方案，不动文件。

```
我要做一个AI文章摘要工具，Web应用。核心功能：
1. 用户输入文章URL
2. 系统抓取文章内容
3. 调用OpenAI API生成3种不同长度的摘要（一句话/短摘要/详细摘要）
4. 存储历史记录，用户可以回看

技术要求：
- Next.js App Router
- SQLite + Drizzle ORM
- Tailwind CSS
- 部署到Vercel

请分析需求，给出完整的技术方案和项目结构。
```

Codex在Plan模式下会输出一份详细的方案，包括目录结构、数据库Schema、API路由设计、前端页面规划。**不要急着说「开始做」**。仔细看方案，挑出你觉得不对的地方。

比如Codex可能会建议用Cheerio做网页抓取，你可以追问：

```
Cheerio能处理JavaScript渲染的页面吗？比如微信公众号文章。
如果不能，有什么替代方案？考虑部署到Vercel的限制。
```

方案在这一步改最便宜。AI给的方案读起来都很合理，部署限制、页面渲染方式这类细节要你追问才会暴露出来。

确认方案后，让Codex生成一份AGENTS.md：

```
基于我们讨论的方案，帮我写一份AGENTS.md。要求：
- 项目根目录一份全局的
- src/app/下前端相关的
- src/lib/下后端和数据库相关的
- 说清楚技术栈、代码风格、文件命名规范
```

后面几个agent并行时，每个都会读AGENTS.md（§05），接口约定写在这里，它们拿到的就是同一份。

阶段二：基础搭建（CLI默认模式）

方案确认了，AGENTS.md也准备好了，现在让Codex动手。退出Plan模式回到默认模式就行：工作区里它可以自由读写、跑命令，装依赖要联网时会来问你。嫌每次问太烦，可以用`/permissions`切到Approve for me，或者启动时加：

```
codex --approve-for-me
```

给它第一个任务：

```
按照AGENTS.md中的方案，初始化项目：
1. 用create-next-app创建Next.js项目
2. 安装依赖：drizzle-orm, better-sqlite3, openai, tailwindcss
3. 创建基本的目录结构
4. 配置Drizzle和数据库Schema
5. 写一个最简单的首页，确认项目能跑起来

先做骨架，不做具体功能。
```

它会自己执行shell命令、创建文件、安装依赖，你在终端里能看到它每一步在做什么。

几个注意事项：

让它跑完再看，不要中途打断。项目初始化是一连串有依赖关系的操作（创建目录→安装依赖→写配置→验证），打断容易导致半成品状态。

跑完之后验证。让Codex执行`npm run dev`，自己打开浏览器看一眼。结构建得整整齐齐、一跑就报错的情况并不少见。

这时候commit。基础骨架搭好、验证能跑，立刻让Codex提交一次：

```
git init && git add . && git commit -m "init: project scaffold with Next.js + Drizzle + SQLit
```

每个里程碑都commit。后面几个agent并行改代码，改崩了要能回退到一个干净的状态。

阶段三：并行开发（App + Worktree）

打开ChatGPT桌面App，切到Codex。App的长处就是几个线程摆在一个窗口里管。（只用CLI的话，开三个终端各跑`codex --worktree`，再用`codex agents`看总览，也能做到。）

我们要开三个并行任务，每个用Worktree模式（§06详细介绍过）跑在自己的独立分支上：

线程	分支名	负责范围
agent 1	feat/frontend-ui	前端所有页面和组件
agent 2	feat/backend-api	API路由+文章抓取+摘要生成
agent 3	feat/database	数据库模型+迁移+CRUD操作

在App里新建一个线程，模式选Worktree，写第一个agent的任务：

```
【agent 1 - 前端UI】

基于AGENTS.md中的设计方案，实现前端所有页面：

1. 首页（src/app/page.tsx）
   - URL输入框（支持粘贴后自动触发）
   - 提交按钮
   - 加载状态动画
   - 摘要结果展示（三种长度切换）

2. 历史记录页（src/app/history/page.tsx）
   - 摘要列表，按时间倒序
   - 点击展开详情
   - 搜索过滤

3. 布局组件（src/app/layout.tsx）
   - 顶部导航栏
   - 响应式布局

用Tailwind CSS，风格简洁干净。不需要实际调用API，
用mock数据把界面先做出来。接口约定：
- POST /api/summarize  body: { url: string }
- GET /api/history
- GET /api/history/:id
```

最后那段接口约定是给并行准备的：**先约定接口，各做各的，最后拼起来**。不约定的话，agent 1可能调`/api/summary`，agent 2实现的是`/api/summarize`，合并时就要改一堆东西。

同样的方式创建agent 2和agent 3：

【agent 2 - 后端API】

实现所有API路由：

1. POST /api/summarize
   - 接收 { url: string }
   - 用fetch抓取文章内容，解析出正文（去掉导航、广告等）
   - 调用OpenAI Responses API（模型用gpt-6-luna，量大便宜）生成三种摘要：
     a. one_line: 一句话总结（<50字）
     b. short: 短摘要（100-200字）
     c. detailed: 详细摘要（300-500字）
   - 存入数据库
   - 返回摘要结果

2. GET /api/history
   - 返回最近50条记录（分页）

3. GET /api/history/[id]
   - 返回单条详情

OpenAI API Key从环境变量OPENAI_API_KEY读取。
数据库操作通过src/lib/db.ts暴露的函数调用。
数据库函数约定：
- insertSummary(data): 插入一条记录
- getSummaries(page, limit): 分页查询
- getSummaryById(id): 查单条

【agent 3 - 数据库】

实现数据库层：

1. Schema定义（src/lib/schema.ts）
   - summaries表：id, url, title, content, one_line, short, detailed, created_at

2. 数据库连接（src/lib/db.ts）
   - 导出Drizzle实例
   - 导出CRUD函数：
     a. insertSummary(data)
     b. getSummaries(page, limit)
     c. getSummaryById(id)
     d. deleteSummary(id)

3. 迁移脚本（drizzle.config.ts + scripts/migrate.ts）
   - 生成并运行迁移

4. 写单元测试验证CRUD操作正确

SQLite文件放在项目根目录的data/目录下。
.gitignore要加上data/*.db

三个agent同时开始工作。你可以在App的任务列表里看到它们各自的进度。每个agent在自己的git worktree里操作，互不干扰。

> **提示：**并行开发最容易出问题的地方是接口对不上。开三个agent之前，先在AGENTS.md里把接口约定写清楚，字段名、类型、返回格式越具体越好。

等agent干活的时候你并不需要干等。可以做几件事：

准备环境变量。创建`.env.local`文件，写入OpenAI API Key。

检查先完成的agent。数据库层通常最先完成，因为代码量最少。它完成后，看一眼schema是否合理、测试是否通过。

准备部署配置。在Vercel上创建项目，配置好环境变量。等代码合并后可以直接部署。

阶段四：集成和测试（CLI + Review）

三个agent都完成了。现在要把三条分支的代码合并到一起。

回到CLI。先看看三个分支的状态：

```
git branch
# * main
#   feat/frontend-ui
#   feat/backend-api
#   feat/database
```

合并顺序是**先底层，再上层**：API依赖数据库，前端依赖API，所以是database → backend-api → frontend-ui。

```
git merge feat/database
git merge feat/backend-api
git merge feat/frontend-ui
```

如果运气好，三次merge都没冲突。但更常见的情况是backend-api和database会有冲突，因为两者都会动`src/lib/`下的文件。遇到冲突，让Codex来解：

```
合并feat/backend-api时遇到冲突，帮我解决。
原则：数据库层以feat/database的实现为准，API层以feat/backend-api为准。
```

合并完成后跑一遍`npm run dev`，多半会报错：三个agent各自用mock数据开发，真正对接时类型可能不匹配、导入路径可能不对。

这时候用Codex的`/review`命令做一次代码审查：

```
/review
```

Codex会检查整个项目的代码，找出类型错误、未使用的导入、不一致的命名等问题。比较典型的问题有：

常见问题	原因	解法
类型不匹配	前端期望的字段名和后端返回的不一样	以AGENTS.md中的接口约定为准，统一修改
导入路径错误	agent各自创建了不同的目录结构	统一目录结构后修改导入
环境变量缺失	后端agent用了硬编码的测试Key	改成 `process.env.OPENAI_API_KEY`
数据库连接重复	API层和数据库层都创建了连接实例	统一用数据库层暴露的实例

逐个修复。每修一个问题，验证一下是否影响其他地方。等所有TypeScript编译错误都消除了，再做一次完整测试：

```
帮我做一次端到端测试：
1. 启动dev server
2. 调用POST /api/summarize，传入一个真实的文章URL
3. 检查返回的摘要是否正确
4. 调用GET /api/history，检查是否有刚才的记录
5. 检查前端页面能否正常展示
```

测试通过后，commit并打tag：

```
git add . && git commit -m "feat: integrate all modules - frontend + api + database"
git tag v0.1.0
```

阶段五：优化和部署

核心功能跑通了，但还有一些收尾工作。这些任务适合丢给云端Codex异步处理：

在网页或桌面App里新建任务，选Work in > Cloud，用这个仓库建好并发布的环境（§07），创建几个任务：

```
【任务1】给项目写README.md，包括：
- 项目简介
- 功能截图占位
- 安装和运行步骤
- 环境变量配置说明
- 部署到Vercel的步骤
```

```
【任务2】优化错误处理：
- API路由统一的错误响应格式
- 前端的错误提示UI
- 文章抓取失败时的降级方案
- OpenAI API调用超时的重试逻辑
```

```
【任务3】添加SEO和性能优化：
- 页面metadata
- OG图片
- 摘要结果页面的SSG
- 图片和字体优化
```

云端Codex的好处是这些任务可以同时跑，每个在独立的沙盒环境里执行，完成后给你提PR。你审查、合并就行。

部署到Vercel：把代码推到GitHub，在Vercel控制台导入项目，配置环境变量，点Deploy。Next.js项目Vercel会自动识别构建配置，通常不需要额外设置。

要注意的是SQLite。Vercel的Serverless函数没有持久的文件系统，SQLite文件存不住，生产环境换成Turso这类托管数据库。让Codex做这个迁移：

```
把数据库从本地SQLite迁移到Turso（libSQL的云端版本）。
要求：
- 本地开发仍然用SQLite文件
- 生产环境用Turso
- 通过环境变量DATABASE_URL区分
```

常见陷阱和解决方案

用Codex做这类多agent并行的项目，常见的坑有这些：

陷阱	现象	解决方案
agent改了不该改的文件	让agent改前端，它顺手把数据库schema也改了	用worktree隔离。每个agent只在自己的分支上操作，合并时人工审查
多agent的修改互相冲突	两个agent都改了同一个配置文件	分清依赖关系，先合并基础模块。共享配置文件只由一个agent负责
云端任务执行失败	提交到云端Codex的任务跑不起来	发布环境前看设置报告，确认依赖装齐、测试能在环境里跑通（§07）
上下文太长导致质量下降	agent干了很多事之后，回答变得不精准	用`/compact`压缩上下文，或者干脆开一个新session。把已完成的工作commit后，新session只需要知道当前状态
接口约定不一致	前端调的接口和后端实现的对不上	在AGENTS.md中定义接口契约，所有agent都读同一份约定
忘记处理边界情况	URL格式错误、文章内容为空、API超时	在初始prompt里就列出要处理的异常情况
不commit就并行开发	三个agent同时改代码，改崩了没法回退	每个里程碑都commit。骨架搭好commit，每个agent完成commit，合并后commit
测试只看能不能跑	页面能打开就算完成，不测边界	写一个端到端测试清单，让Codex逐项验证。包括正常流程和异常流程

三条原则

不要一次给AI太大的任务。「帮我做一个AI文章摘要工具」太大。「帮我实现POST /api/summarize路由，接收URL参数，抓取正文，调用gpt-6-luna生成三种长度的摘要」就具体得多。任务越具体，输出越可控。

每一步都验证。骨架搭好，验证一次；数据库建好，跑一遍测试；API写好，curl测一下；前端做好，打开浏览器看一眼。每一步验证的成本很低，攒到最后一起查，成本就高了。

自己判断输出好不好。AI生成的代码读起来总是很有道理，它很少主动说「我不确定这样做对不对」。这个数据库Schema合理吗？错误处理够不够？API设计后面好扩展吗？这些要你来判断。

> **花叔的经验：**我不写代码，小猫补光灯从第一行到最后一行都是AI写的。我花时间的地方是想清楚要做什么、不做什么，以及每一版做出来以后，用起来到底对不对。

§10 Codex和Claude Code：两个都用时怎么分工

Using Codex and Claude Code Together

半年前两家还能列出一长串「对方没有的功能」，现在这张清单短了很多。这一章按10月20日两家官方文档的内容，对一遍模型、功能和价格，再给几种实际的组合用法。

功能清单已经很像了

5月版书里，§08把`/goal`和定时任务写成Codex的特色，§10还把hooks放进了Claude Code的独有项。到10月，这几样两边都有了：

能力	Codex	Claude Code
入口	CLI、ChatGPT桌面App、IDE扩展、网页、手机、云端	CLI、桌面App、IDE扩展、网页、手机
持续目标	`/goal`（0.128.0，4月30日）	`/goal`（2.1.139，5月11日）
定时和循环	Scheduled tasks（App和网页）	`/loop`（会话内）、Routines（云端，研究预览）
项目规则	AGENTS.md	CLAUDE.md；2.1.277起没有CLAUDE.md时也读AGENTS.md
扩展	Skills、插件、MCP、hooks	Skills、插件市场、MCP、hooks
多agent	子agent、App里多线程	子agent、`claude agents`多会话视图、Agent Teams（实验性）

剩下的差别多在细节里。hooks两边都有，Claude Code的事件列表长得多（官方文档列了三十个左右，Codex是十几个）。定时任务Codex挂在ChatGPT的App和网页上，能和已连接的Gmail、Slack这些应用打通；Claude Code分成会话内的`/loop`、桌面App的定时任务和云端的Routines（研究预览），Routines可以用定时、API或GitHub事件触发。Codex这边还有§07讲的那一套云端环境：可复用环境、网络密钥、从Slack和Linear派活。

所以选哪个，功能清单已经帮不上太多忙，要看的是三件事：模型、价格和额度、你的团队已经在哪个生态里。

10月的模型对照

两家这半年都整代换了模型。Codex这边是GPT-6系列（§04），Claude那边是9月初的Fable 5.1，和9月下旬到10月初陆续发布的Claude 5.5系列。按两家官方模型页：

定位	OpenAI	API价格（输入/输出）	Anthropic	API价格（输入/输出）
最强	GPT-6 Astra	10 / 50美元	Claude Fable 5.1	10 / 50美元
日常主力	GPT-6.1 Sol（Codex默认）	2 / 10美元	Claude Opus 5.5（官方建议的起点）	4 / 20美元
中档	—	—	Claude Sonnet 5.5	2 / 10美元
快且便宜	GPT-6 Luna	0.1 / 0.5美元	Claude Haiku 5.5	0.1 / 0.5美元起

价格是每百万token。上下文Claude这四个模型都标1M；Codex里能用多少由客户端决定，§03的`/status`截图里GPT-6.1 Sol显示828K。

基准分数怎么看

两家这一轮都换了基准，SWE-bench Verified和Terminal-Bench 2.0在新发布页里已经不报了，换成了Terminal-Bench 4.0、SWE-bench Pro、DeepSWE这些更难的。下面是Anthropic在Opus 5.5发布页上放的对照（GPT-6 Astra的数字Anthropic注明是「as reported by OpenAI」）：

基准	Claude Opus 5.5	Claude Fable 5.1	GPT-6 Astra
Terminal-Bench 4.0	66.4%	55.8%	57.9%
FrontierCode v1.1 Main	54.4%	50.3%	53.3%

这张表有三处要留意：

1. **这是各家自报的分数。**每个模型取的是各自最高档位的成绩（Opus 5.5是xhigh，GPT-6 Astra是high），Astra的数字来自OpenAI自己的测试，两边不是在同一套环境里跑的。两家页面对同一个模型的数字有时也对不上，引用时只引一家的页面。
2. **Codex的默认模型不在表里。**GPT-6.1 Sol的官方发布页没有给Terminal-Bench 4.0分数，OpenAI给的说法是它在DeepSWE v1.1上和GPT-6 Astra持平，成本大约是五分之一。
3. **Anthropic给的SWE-bench Pro（Opus 5.5是89.9）OpenAI没有对应的数。**两家各挑了自己的主基准，没法逐项对齐。

在这两项上，三个模型之间相差1到9个百分点，而GPT-6.1 Sol的价格只有Opus 5.5的一半。对日常编码来说，成本、额度和工作流顺不顺手，往往比基准上那几个点更影响体验。

几种实际的组合用法

一份规则文件，两边共读。Claude Code从2.1.277起会读AGENTS.md，前提是同一条路径上没有CLAUDE.md（附录C有细节）。新项目只写一份AGENTS.md，两边的agent拿到的是同一套构建命令、测试命令和禁区。

一个写，另一个审。一家的agent改完代码，先别提交，在终端里让另一家审一遍。比如Claude Code改完，跑：

```
codex review --uncommitted
```

它会审工作区里还没提交的改动。两个不同的模型看同一份diff，容易抓到对方的盲区。反过来也一样，Codex写完的PR可以让Claude Code审。

按活的形态分。要丢到云端跑、电脑关了也继续的活，从GitHub issue、Slack、Linear里直接派的活，Codex的云端环境更现成（§07）。需要很细的hooks去拦截、记录、改写agent行为的活，Claude Code的事件更全。团队已经用ChatGPT Business的，Codex和ChatGPT Work共用一份订阅，不用多开一个账号。

额度分摊。两个入门档加起来每月40美元。一家5小时窗口用完了，切到另一家接着做，项目规则是同一份AGENTS.md，切过去不用重新交代。批量、重复的小活交给GPT-6 Luna这类便宜模型，额度能撑很久。

从Claude Code搬过来。Codex的`/import`能把Claude Code的设置、项目和近期会话导进来（§04）。它会直接改`~/.codex/config.toml`，先备份再导。

怎么跟上变化

这本书5月版到10月版，半年里两家各换了一代模型，Codex CLI从0.130走到0.163，产品形态也从独立App并进了ChatGPT桌面App。你读到这里的时候，书里的一些细节可能又变了。几个习惯：

- **看changelog，别看二手解读。**两家都有官方changelog，每周扫一眼就够。
- **以本机为准。**命令和参数先跑`codex --help`、`/status`、`/model`看一眼，本书§04那几个被删掉的参数（`--full-auto`、`--api-key`）就是这么发现的。
- **两个都装上，用自己的项目试。**基准测的是标准任务，你每天面对的是自己的代码库和自己的习惯。同一个活两边各跑一次，比任何对比表都准。

推荐资源

资源	地址	用途
Codex官方文档	learn.chatgpt.com/docs	功能参考、配置、最佳实践
Codex更新日志	learn.chatgpt.com/docs/changelog	每次更新的一手说明
Codex CLI仓库	github.com/openai/codex	源码、release、issue
Codex定价	learn.chatgpt.com/docs/pricing	套餐、额度、积分费率
Claude Code文档	code.claude.com/docs	官方功能参考
Claude Code更新日志	code.claude.com/docs/en/changelog	版本更新一手
Claude定价	claude.com/pricing	Pro、Max套餐

附录A 命令速查表

Command Reference

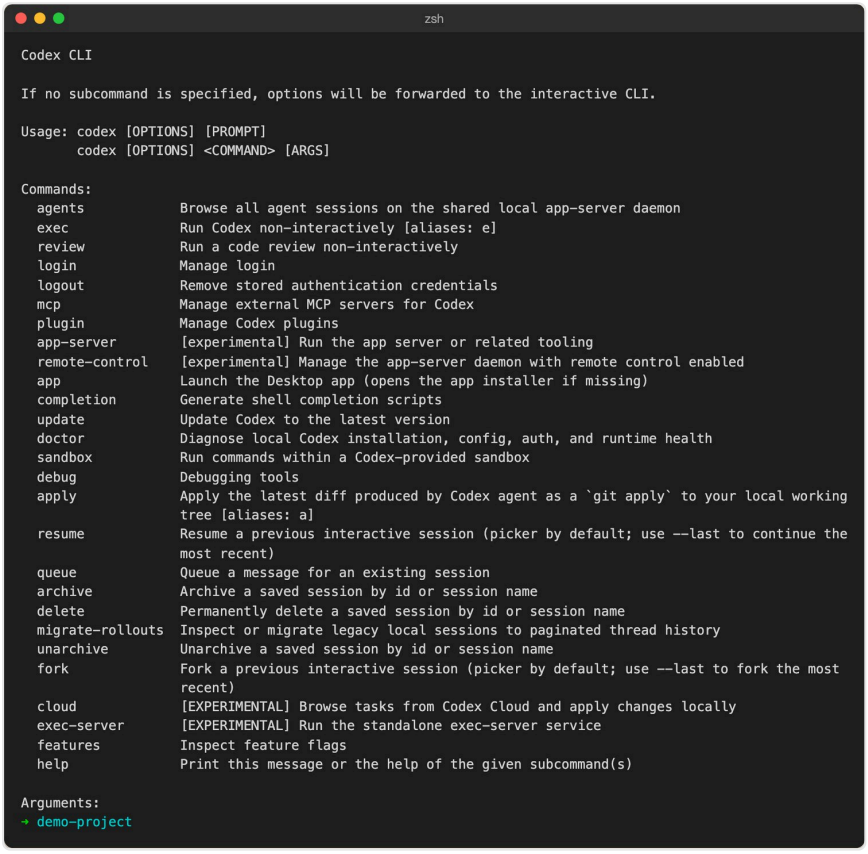
按Codex CLI 0.163.0整理：子命令和参数来自本机的`codex --help`，斜杠命令来自会话里逐行翻出来的菜单。新版本可能有增减，以你本机为准。

CLI子命令

命令	用途
`codex`	启动交互界面
`codex exec`（`e`）	非交互执行；子命令`exec resume`、`exec review`、`exec fork`
`codex review`	非交互代码审查：`--uncommitted` / `--base 分支` / `--commit SHA`
`codex resume`	恢复之前的会话，默认弹选择器，`--last`接上一次，`--all`不限目录
`codex fork`	从之前的会话分叉出一条新线
`codex archive` / `unarchive` / `delete`	按ID或会话名归档、取消归档、永久删除会话
`codex queue --thread <ID> --message "..."`	给一个已有会话排队一条消息
`codex agents`	浏览共享后台服务上的所有会话
`codex login` / `logout`	登录管理：`--device-auth`、`--with-api-key`、`--with-access-token`；`codex login status`看登录状态
`codex mcp`	管理MCP服务器：list / get / add / remove / login / logout
`codex plugin`	管理插件：add / list / marketplace / remove
`codex app`	打开桌面App，没装就打开安装程序
`codex cloud`	（实验）云端任务：list / exec / status / diff / apply
`codex apply <TASK_ID>`（`a`）	把Codex最近一次产出的diff用`git apply`应用到本地
`codex update`	自更新
`codex doctor`	体检安装、配置、登录、运行环境
`codex features`	查看功能开关（`codex features list`）
`codex sandbox`	在Codex的沙盒里跑一条命令：`codex sandbox -- ls`
`codex completion`	生成shell补全脚本（zsh / bash / fish）

命令	用途
`codex remote-control`	（实验）开启远程控制的后台服务：start / stop / pair（打印手机配对码）
`codex app-server`、`exec-server`	（实验）给集成方用的服务进程
`codex migrate-rollouts`	把旧的本地会话迁移到新的线程历史格式
`codex debug`	调试工具

已经没有的：`codex mcp-server`（9月5日删除）。



> `codex --help` 的前40行，本表的子命令清单以它为准。

全局参数

参数	用途
`-m, --model`	指定模型，如`gpt-6.1-sol`、`gpt-6-astra`、`gpt-6-luna`
`-c, --config key=value`	临时覆盖一项配置，如`-c model_reasoning_effort=high`
`-p, --profile 名字`	在基础配置上叠加`~/.codex/名字.config.toml`
`-s, --sandbox`	`read-only` / `workspace-write` / `danger-full-access`
`-a, --ask-for-approval`	`on-request` / `never`
`--approve-for-me`	审批请求先交给自动审查（Auto-review）
`--dangerously-bypass-approvals-and-sandbox`（`--yolo`）	跳过所有确认、不用沙盒，只在外面另有隔离时用
`--dangerously-bypass-hook-trust`	本次运行不要求hooks先被信任，只给已审过hook来源的自动化用
`-C, --cd 目录`	指定工作根目录
`--add-dir 目录`	额外的可写目录
`--worktree`	在一个新的受管git worktree里开会话
`-i, --image 文件…`	附带图片，可以给多个
`--search`	开实时Web搜索
`--enable` / `--disable 功能`	临时开关某个功能
`--strict-config`	配置有问题时直接报错
`--no-alt-screen`	不占满屏幕，保留终端滚动历史
`--no-daemon`	不连共享的后台服务
`--oss`、`--local-provider`	用本地开源模型

参数	用途
`--remote`、`--remote-auth-token-env`	连远程的app-server

`codex exec` 另有：`--json`（输出JSONL事件）、`-o, --output-last-message 文件`、`--output-schema 文件`、`--ephemeral`（不落盘会话）、`--skip-git-repo-check`、`--ignore-user-config`、`--ignore-rules`。

已经失效的：`--full-auto`、`--api-key`、`-a untrusted`、`-a on-failure`（§04有截图和替代写法）。

TUI斜杠命令（0.163.0本机菜单，共56个）

命令	用途
模型与权限	
`/model`	选模型和推理档位
`/fast`	快速档：约2倍速度，用量更多
`/permissions`	选Codex能做什么（四个预设）
`/approve`	放行一次最近被自动审查拒掉的动作
`/plan`	切到Plan模式
`/goal`	设置或查看长任务的目标（edit / pause / resume / clear）
`/experimental`	开关实验功能
`/daybreak`	了解网络安全工作的更宽访问权限
对话与会话	
`/new`	在当前会话里开一个新对话
`/clear`	清屏并开始新对话
`/compact`	总结对话，避免撑满上下文
`/recap`	马上总结当前对话
`/side`	在一个临时分叉里开旁支对话
`/fork`	分叉当前对话
`/resume`	恢复一个保存的对话
`/rename`	重命名当前线程
`/archive` / `/delete`	归档/永久删除当前会话
`/export`	把对话导出成markdown
`/copy`	复制最后一条回复或其中一部分

命令	用途
`/import`	从Claude Code导入设置、当前项目和近期对话（回车即执行）
`/app`	在桌面App里继续这个会话
`/exit`	退出
`/feedback`	把日志发给维护者
`/logout`	退出登录
代码与工作区	
`/diff`	看git diff（含未跟踪的文件）
`/review`	审查当前改动、找问题
`/mention`	提及一个文件
`/init`	生成AGENTS.md
`/cd` / `/pwd`	切换/显示当前工作目录
`/worktree`	在一个新worktree里开始或继续对话
`/ide`	带上IDE里的选区、打开的文件等上下文
`/ps` / `/stop`	列出/停掉后台终端
agent与扩展	
`/agents`	打开agent命令中心
`/subagents`	在本会话的子agent之间切换
`/skills`	浏览和使用skills
`/plugins`	浏览插件
`/mcp`	列出MCP工具；`/mcp verbose`、`/mcp login 名字`
`/hooks`	查看和管理hooks
`/memories`	设置记忆的使用和生成

命令	用途
界面与设置	
`/config`	可搜索的设置面板（0.163.0新增）
`/keymap`	重新绑定快捷键
`/tui`	选下次启动用全屏还是Scrollback界面
`/theme`	选语法高亮主题
`/statusline` / `/title`	设置状态栏/终端标题显示哪些信息
`/vim`	输入框的Vim模式
`/raw`	原始滚动模式，方便复制
`/voice`	开关语音，`/voice settings` 选声音
`/pets`	选择或隐藏终端宠物
账号与诊断	
`/status`	当前会话的配置和token用量
`/usage`	账号用量，或兑换一次额度重置
`/daemon`	管理本地后台服务
`/warnings`	看保留下来的警告和诊断信息
`/feedback`	把日志发给维护者
`/logout`	登出

菜单里没有的：`/personality`、`/reasoning`、`/agent`（单数）。

快捷键（默认键位）

按键	作用
`Enter`	发送；它在干活时，插一句话调整当前这一轮
`Tab`	排队一条消息，这一轮结束后再处理
`Esc`	打断当前这一轮
`Esc Esc`	输入框为空时，回溯编辑之前的消息
`@`	模糊搜索并引用文件
`!命令`	直接跑本地shell命令
`Ctrl+T`	展开完整的转录记录
`Ctrl+G`	用外部编辑器写长prompt
`Ctrl+O`	复制
`Ctrl+L`	清屏（不清对话）
`Ctrl+R`	搜索输入历史
`Ctrl+/`	开关旁支对话
`Alt+R`	开关原始输出
`Alt+,` / `Alt+.`（或 `Shift+↓` / `Shift+↑`）	降低/提高推理档位
`Ctrl+X`	leader键（0.163.0新增，可在 `/keymap` 里改）
`F8` / `F9`	开关语音/语音静音
`Ctrl+C`	退出

config.toml常用配置项

```
# ~/.codex/config.toml

model = "gpt-6.1-sol"
model_reasoning_effort = "medium"   # low / medium / high / xhigh，另有 max、ultra

sandbox_mode = "workspace-write"
approval_policy = "on-request"      # 或 "never"；"untrusted" 已退役，写了会启动失败
# approvals_reviewer = "auto_review"  # 打开自动审查

web_search = "cached"               # cached / indexed / live / disabled

review_model = "gpt-6.1-sol"        # 审查用的模型（可选）

[agents]
max_concurrent_threads_per_session = 6

[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

# 分场景的配置：另写一个 ~/.codex/work.config.toml，用 codex -p work 叠加
# 5月版书里的 [profiles.xxx] 写法从0.134.0起不再支持
```

AGENTS.md基本写法

```
# AGENTS.md

## 项目说明
这是一个Next.js的Web应用……

## 编码规范
- 使用TypeScript strict模式
- 组件用函数式写法
- 测试用Vitest

## 常用命令
- npm run dev：启动开发服务器
- npm test：运行测试
- npm run build：构建生产版本

## Code Review Rules
- 新增API路由必须有认证中间件
```

建议：AGENTS.md和CLAUDE.md写法很像，都是用自然语言写项目信息和规范。两个文件可以在同一个项目里共存，也可以只留一份AGENTS.md，再让Claude Code读它（具体做法见附录C）。

附录B 定价与套餐

Pricing and Plans

按OpenAI官方定价页（learn.chatgpt.com/docs/pricing）2026年10月20日的内容整理。价格和额度变得很快，下单前以官方页面为准。

半年里变了什么

时间	事件
2026年4月	Codex按token计费；GPT-5.5进入Codex（5月版书的基线）
7月9日	Codex并入ChatGPT桌面App，产品名变成「ChatGPT Work and Codex」，Work和Codex共用用量
8月31日	GPT-5.4和GPT-5.4 mini对ChatGPT登录的Codex下线
9月14日	GPT-5.3-Codex-Spark退役
9月3日	GPT-6 Astra发布
9月下旬	GPT-6 Sol、GPT-6 Luna上线；9月29日GPT-6.1 Sol上线并成为CLI默认
10月14日	GPT-5.5对ChatGPT登录的Codex退役（API和API Key会话不受影响）

个人和团队套餐

套餐	价格	和Codex有关的要点
Free	0	桌面App里用GPT-6 Luna，标准速度，逐步放量中；没有图片生成
Go	8美元/月	同Free，桌面App里的GPT-6 Luna
Plus	20美元/月	CLI、IDE扩展、网页、iOS、云端集成；GPT-6.1 Sol和GPT-6 Luna；可以另买积分
Pro	100 / 200 / 500美元/月三档	官方写的是「Pro plans currently have no five-hour limit」；Ultrafast只有500美元那档有
Business	年付每人20美元/月（2人起），月付25美元	更大的云端虚拟机、积分、SSO和MFA
Enterprise / Edu	联系销售	优先处理、SCIM、EKM、RBAC、合规API、数据驻留
API Key	按量	CLI、SDK、IDE能用；云端那一套（GitHub审查、Slack等）不能用

Free和Go的Codex官方只写了桌面App，能不能在CLI里用没有明确说法。Pro三档的额度，官方只明确了一个：Pro 500是Plus的25倍（DevDay回顾文章）。Pro 100是Plus的5倍，这是OpenAI 4月推出这一档时在X上说的；Pro 200官方帮助页只说「比100多」，媒体报道是约10倍。Pro 200在9月底调过一次：9月22日到29日之间已经订阅的人，旧额度保留到10月29日，10月30日起统一降到新的额度，价格不变。以账号里实际显示为准。

Plus和标准Business：5小时窗口能发多少条

官方给的是估算区间，不是固定上限，本地消息和云端任务共用这份额度，另外可能还有周限额：

模型	5小时内本地消息（估算）
GPT-6 Astra	5–45
GPT-6.1 Sol	15–160
GPT-6 Sol	15–150
GPT-6 Luna	350–3,000

区间这么宽，是因为一条消息花多少取决于任务大小、上下文长短、推理档位。看自己还剩多少，会话里用 `/status`（周额度）和 `/usage`，网页上是 `chatgpt.com/codex/settings/usage`。

积分（Credits）费率

额度用完或者在Business、Enterprise这类按积分计的工作区，用的是积分。每百万token、标准速度：

模型	输入	缓存输入	输出
GPT-6 Astra	250	25	1,250
GPT-6.1 Sol	50	2.5	250
GPT-6 Sol	50	5	250
GPT-6 Luna	2.5	0.25	12.5
GPT-5.6 Sol	100	10	500
GPT-5.6 Terra	50	5	300
GPT-5.6 Luna	5	0.5	30

几个乘数：

- **Fast**：订阅额度按2.5倍算，购买的积分按2倍算。
- **Ultrafast**（GPT-6 Astra和GPT-6.1 Sol，只有Pro 500美元档和合格的Enterprise、Edu）：订阅额度8倍，积分6倍。
- **图片生成**：大约按普通消息的3到5倍消耗。
- **语音**：桌面App的语音按每分钟0.05美元计入Codex额度（积分制工作区是每分钟1.25积分）。

API价格

用API Key登录Codex时按这个付费（每百万token）：

模型	输入	缓存输入	输出
GPT-6.1 Sol	2美元	0.1美元	10美元
GPT-6 Astra	10美元	1美元	50美元
GPT-6 Luna	0.1美元	0.01美元	0.5美元

单次请求输入超过272K token时，整次请求按2倍输入价、1.5倍输出价算。API模型页上这三个模型的上下文都标1,050,000，但Codex里实际能用多少由客户端决定：§03的`/status`截图里，默认配置下GPT-6.1 Sol显示的是828K。

省额度的办法

- **按任务选模型。**日常编码用GPT-6.1 Sol；重复、批量、简单的活用GPT-6 Luna，便宜两个数量级；只有最难的才上GPT-6 Astra。
- **推理档位别一律拉满。**GPT-6.1 Sol默认是Low，多数任务够用；Max、Ultra的用量涨得最快，Ultra会开多个agent并行。
- **子agent配便宜的模型。**`[agents]`里的`default_subagent_model`设成`gpt-6-luna`。
- **长任务设预算。**`/goal`从0.148.0起可以配token预算，碰到用量上限会自动停（§08）。
- **Fast按需开。**赶时间才开，平时关着。

与Claude Code订阅的对照

从Claude Code过来的读者常问两边怎么比。按两家官方页面10月20日的内容：

档位	ChatGPT（Codex）	Claude（Claude Code）
入门	Plus 20美元/月	Pro 20美元/月（年付折合17美元）
重度	Pro 100 / 200 / 500美元/月	Max 5x 100美元/月、Max 20x 200美元/月
团队	Business，年付每人20美元/月起	Team、Enterprise

API价格（每百万token，输入/输出）：

OpenAI	价格	Anthropic	价格
GPT-6 Astra	10 / 50美元	Claude Fable 5.1	10 / 50美元
GPT-6.1 Sol	2 / 10美元	Claude Opus 5.5	4 / 20美元
—	—	Claude Sonnet 5.5	2 / 10美元
GPT-6 Luna	0.1 / 0.5美元	Claude Haiku 5.5	0.1 / 0.5美元起

两家最强的模型同价。Anthropic官方把Opus 5.5定为大多数任务的起点，它的价格是GPT-6.1 Sol的两倍；和GPT-6.1 Sol同价的是Sonnet 5.5。订阅额度两家都不按token公开，算不出谁更划算，最实际的办法是各订一个月入门档，用自己的真实项目跑一样的活，看谁先碰到上限。

附录C 常见问题

Frequently Asked Questions

安装与登录

Q：装Codex CLI需要先装Node.js吗？

不用。官方首推独立安装器：macOS和Linux用`curl -fsSL https://chatgpt.com/codex/install.sh | sh`，Windows在PowerShell里用`irm https://chatgpt.com/codex/install.ps1 | iex`。用npm装（`npm install -g @openai/codex`）才需要Node.js。Homebrew要写`brew install --cask codex`。装完跑`codex --version`和`codex doctor`确认。

Q：`codex --api-key`报错？

这个参数已经没有了。用API Key登录要通过标准输入：`printenv OPENAI_API_KEY | codex login --with-api-key`。没有浏览器的机器上用ChatGPT账号登录，用`codex login --device-auth`。

Q：免费账号能用Codex吗？

Free和Go套餐现在有Codex，官方写的是在桌面App里用GPT-6 Luna，还在逐步放量；CLI能不能用，官方没有明确说法。想稳定用CLI、IDE扩展和云端，从Plus起步。

Q：已经有ChatGPT账号，还要单独注册Codex吗？

不用。Codex是ChatGPT订阅的一部分，用同一个账号登录，用量和ChatGPT Work共用。

网络

Q：在国内用要注意什么？

CLI跑在你本机，但要能稳定访问OpenAI的服务。走代理时，注意启动Codex的那个终端里有没有带上代理的环境变量；本书实测时就遇到过一次没带代理导致请求全部超时，补上后又出现一次「Falling back from WebSockets to HTTPS transport」，大约一分钟后恢复。

Q：云端任务一直不动？

先看三处：环境是不是已经Publish了（没发布的环境起不了新任务）；用量是不是快到上限（`chatgpt.com/codex/settings/usage`）；GitHub仓库的授权还在不在。

模型

Q：GPT-5.5去哪了？

10月14日起对ChatGPT登录的Codex退役，GPT-5.4在8月31日、GPT-5.3-Codex-Spark在9月14日就已经下线。用API Key登录的会话不受影响。官方给的替换是：Plus、Pro、Business、Enterprise、Edu换`gpt-6.1-sol`，Free和Go换`gpt-6-luna`。

Q：现在的几个模型怎么选?

模型	官方定位	适合
GPT-6.1 Sol	当前主力，CLI默认	日常编码和大多数任务
GPT-6 Astra	最强	最难、最要质量的任务；官方建议从低档位起步
GPT-6 Luna	快且便宜	重复、批量、聚焦的小任务，子agent
GPT-6 Sol、GPT-5.6系列	上一代	还能选，新项目没必要

Q：默认推理档位是多少?

本机0.163.0里，GPT-6.1 Sol默认Low，Astra和Luna默认Medium。5月版说的默认xhigh已经不对了。在`/model`里选档位时，Enter是改默认，s只改本次会话。

Q：上下文到底有多大?

API模型页上GPT-6.1 Sol、Astra、Luna都标1,050,000，但Codex里能用多少由客户端决定。本书实测：默认配置下GPT-6.1 Sol的`/status`显示828K；手动设过`model_context_window`的配置会显示别的数。以你自己`/status`里的数字为准。上下文越长，模型对细节的记忆越不可靠，大代码库还是让它按需读文件更稳。

权限与安全

Q：`--full-auto`没了，现在怎么让它少打断我?

默认（Ask for approval）已经是「工作区里随便改，越界才问」。想再少一点打断，`/permissions`选Approve for me，或者启动时加`--approve-for-me`：越界请求先交给自动审查，只有它判为可能不安全的才来问你。被它拒掉的，用`/approve`放行一次。

Q：Full Access能开吗?

只在外面另有一层隔离（容器、虚拟机）时开。Windows上尤其谨慎：上半年社区有多起Full Access下删掉工作区以外文件的报告，其中GitHub #18509到10月20日仍是open（根因未确认）。不可逆的操作前先提交或备份。

Q：配置里写了`approval_policy = "untrusted"`，现在启动报错?

`untrusted`已经退役，配置里带着它客户端会起不来。改成`"on-request"`，或者用自动审查。

从Claude Code过来

Q：能把Claude Code的设置直接搬过来吗?

能。CLI里`/import`，桌面App里Settings > Import，可以导入设置、当前项目和近期会话（App里还支持Claude Cowork和Cursor，可以开自动同步）。**它会直接改你的`~/.codex/config.toml`，并把旧会话加进历史，本书实测时CLI里回车就直接导入了，没看到确认。**先备份config.toml再导。导进来的会话可以用`codex archive 会话ID`归档，之后还能`codex unarchive`。

Q：CLAUDE.md和AGENTS.md能共存吗?

能，但要知道Claude Code怎么读这两个文件。Claude Code从2.1.277起原生读AGENTS.md，规则是：从当前目录往上，只要有一个CLAUDE.md（或CLAUDE.local.md），它就只读CLAUDE.md，同一条路径上的AGENTS.md整个不读；一个CLAUDE.md都没有，才读AGENTS.md。这个行为可以在Claude Code的`/config`里Project instructions一项改，默认值是`claude-md-or-agents-md`，改成`claude-md-and-agents-md`可以两个一起读。要写进配置文件的话，它在用户设置的`pluginConfigs`下面，写在项目设置里不生效。另外，`~/.claude/CLAUDE.md`不算「路径上有CLAUDE.md」，项目里的`.claude/CLAUDE.md`和`CLAUDE.local.md`算。

所以两边一起用，最省事的做法是只留AGENTS.md：Codex读它，新版Claude Code也读它。如果项目里已经有CLAUDE.md，要么把内容并进AGENTS.md再删掉CLAUDE.md，要么在CLAUDE.md里写一行`@AGENTS.md`导入。别在同一个仓库里一部分目录放AGENTS.md、一部分放CLAUDE.md，Claude Code那边会有规则静默失效。

Skills与扩展

Q：Skill不生效?

1. 目录结构对不对：要有`SKILL.md`，开头有name和description。
2. 放的位置对不对：项目里是`.agents/skills/`，个人是`~/.agents/skills/`。
3. 会话里`/skills`看它在不在列表里。
4. description是不是写清楚了什么时候该用；不放心就显式写`$skill-name`。
5. 有没有在`[[skills.config]]`里被设成了`enabled = false`。

Q：MCP服务器连不上?

会话里`/mcp verbose`看详细状态，`/warnings`看启动时的报错。需要OAuth的用`/mcp login 名字`。启动慢的服务器在`[mcp_servers.名字]`里把`startup_timeout_sec`调大（默认10秒）。还不行就跑一遍`codex doctor`。

Q：hook没执行?

新的或改过的hook要先审核信任才会跑，`/hooks`里看它是不是在等审核；项目里的hook只在你信任了这个项目之后才加载。

云端

Q：云端任务失败，常见原因?

1. **环境没准备好**：发布前看设置报告，确认测试能在环境里跑通。
2. **缺环境变量或密钥**：在环境配置里补；给外部服务的凭据用Network secret。
3. **要联网但没开**：环境默认不联网，按需开Package managers预设或加具体域名。
4. **任务描述太含糊**：云端不会中途来问你，相关文件、预期行为、怎么验证都写进去。

Q：云端的结果怎么拿到本地?

开了PR就在GitHub上合并再`git pull`；也可以用`codex cloud apply 任务ID`直接把diff应用到本地。

用久了会遇到的

Q：额度掉得快?

1. 看模型：GPT-6 Astra在Plus的5小时窗口里只有5到45条，Luna有350到3,000条。
2. 看档位：Max、Ultra、Fast都会成倍消耗。
3. 看后台：自动代码审查、定时任务、子agent都在花额度。`/usage`能看历史。

Q：找不到之前的对话?

`codex resume --all`不限目录列出全部；归档过的用`codex unarchive`。关键结论别只放在对话里：让它把决策和待办写进项目里的一个markdown，或者用`/export`导出。

Q：它为什么不先问问题就开干?

默认模式下它倾向于边说边做。想先看方案，用`/plan`切到Plan模式；或者在AGENTS.md里写一句：

```
开始任何非trivial任务之前，先用2-3个问题确认需求范围和验收标准，得到我的回答后再开干。
```

OpenAI Codex从入门到精通

面向工程师与产品经理的Codex全形态实战指南



花叔 · AI Native Coder

我一行代码都不会写，却用AI做出了AppStore付费榜Top 1的小猫补光灯，写了多本技术书。

所有产品全部AI写的，我只负责想清楚要做什么。我始终相信：AI生成内容不稀缺，稀缺的是判断力。

B站：花叔v · 公众号：花叔 · X：花叔 · YouTube · GitHub · 官网

Created by花叔 · 2026年4月初版 · 10月更新

所有橙皮书免费下载：huasheng.ai/orange-books